VIT VELLORE

COMPUTER NETWORKS

PROJECT REPORT

SMART-RUDP: A RELIABLE TRANSPORT PROTOCOL BUILT ON UDP WITH LEARNING-BASED CONGESTION CONTROL

Design, Implementation and Validation in C, with a Multi-Site OSPF Network Design and Wireshark Analysis

Submitted by

Harshavardhan Mallela (24BAI0274)

Rohit Arya (24BCE0819)

Shloak Sinha (24BDS0378)

Faculty: Anuradha G

Source code: github.com/Capsturrrr/rudp

8 October 2026

ABSTRACT

The Transmission Control Protocol (TCP) provides reliable, ordered and congestion-aware delivery over an unreliable network, but its mechanisms are usually studied only in theory. This project designs and implements RUDP (Reliable UDP), a custom transport-layer protocol written in C on top of UDP sockets, which reproduces the core mechanisms of TCP: a three-way handshake, sequence numbers with a 16-bit checksum, cumulative acknowledgments, retransmission on timeout and on three duplicate acknowledgments, a Go-Back-N sliding window, and additive-increase/multiplicative-decrease (AIMD) congestion control with slow start and congestion avoidance.

The project is extended in two directions. First, Smart-RUDP replaces the fixed AIMD rule with a tabular Q-learning congestion controller and compares it with AIMD on three emulated paths (5G-like, lossy wireless and satellite-like); the tabular agent matched but did not beat the rule-based controller on completion time, while a neural (deep Q-network) agent added later beat it on all three paths in the C transport (Section 8.5). Second, the protocol is placed in a multi-site network of four OSPF routers, two VLANs, a server LAN and a branch LAN built in Cisco Packet Tracer, and its traffic is analysed in Wireshark with a custom dissector.

The base implementation (Phases 0 to 7) is 673 lines of C in four source files; the later extensions add the Smart-RUDP sender, the chat programs, an impairment proxy and the tests. A test message of 2,442 bytes, sent as 62 packets, was delivered completely at every simulated loss level from 0% to 40%. Over the same range the transfer time rose from 108 ms to 20,236 ms and the final congestion window fell from 18.71 to 2.00 packets, which is the behaviour AIMD is designed to produce. Linux tc/netem loss injection and packet capture were used to check the protocol against real operating-system packet loss. The report also records the engineering problems met during the work, with screenshots, and how each was fixed.

The final system uses Selective Repeat with a 256-packet window and an ACK bitmap, a CRC-16 checksum, an adaptive retransmission timer, and, for the neural agent, a slow-start phase, pacing and a hand-over to AIMD under heavy loss. It was measured with a real clock, on real UDP sockets through an impairment proxy, and in a virtual-time version of the same C sender that reproduces the real runs in all 15 calibration cells. Against a tuned AIMD baseline the neural agent needs 15% to 53% less time on 5G-like, lossy, satellite-like and fast paths and is not significantly different on a path with 10% random loss; on 100 random held-out paths it needs 0.53 times the time of tuned AIMD and is never more than 20% slower. Against tuned CUBIC it is significantly faster on the lossy and satellite-like paths and equal elsewhere. It is also a more aggressive neighbour than AIMD or CUBIC (68% of a shared link against tuned AIMD). The report records the defects the testing found (a lost handshake ACK that hung the connection, a weak checksum, spurious timeouts that had inflated the apparent advantage) and the ideas that did not work.

Keywords: reliable transport, UDP, TCP, AIMD, congestion control, Q-learning, OSPF, Cisco Packet Tracer, Wireshark.

1. INTRODUCTION

Applications that send data across a network choose between two transport protocols. The User Datagram Protocol (UDP) [8] is simple and fast but gives no guarantee that a datagram arrives, arrives once, or arrives in order. TCP adds all of these guarantees, and also slows its sending rate when the network is congested, but it lives inside the operating-system kernel, which makes its behaviour difficult to observe or change.

The cost of ignoring congestion became clear in October 1986, when the Internet suffered its first congestion collapse and throughput between Lawrence Berkeley Laboratory and UC Berkeley fell from 32 Kbps to 40 bps [1]. TCP congestion control was developed in response. A sender that only retransmits lost data makes such a collapse worse, so reliability and congestion control must be designed together.

This project asks whether TCP's core ideas can be rebuilt from first principles on top of UDP, and whether the result can be shown to work under packet loss. The answer is RUDP, a small user-space protocol written in C and tested on Linux with both a built-in loss simulator and kernel-level network emulation.

1.1 Objectives

- Define a compact packet format with sequence numbers, flags and an integrity check.
- Establish connections with a three-way handshake and close them with a FIN exchange.
- Deliver data reliably and in order using acknowledgments, timeouts and fast retransmit.
- Control the sending rate with AIMD congestion control and make the congestion window visible in logs and graphs.
- Validate the protocol under loss from 0% to 40% and under real packet loss injected by tc/netem.
- Extend the protocol with a Q-learning congestion controller (Smart-RUDP) and compare it honestly with AIMD under emulated 5G, wireless and satellite conditions.
- Build a four-router OSPF network with VLANs, DHCP, an access list and a failover test in Cisco Packet Tracer, and analyse RUDP traffic with Wireshark.
- Record every mistake made during development and how it was corrected.

1.2 Application area

RUDP belongs to user-space reliable transport over UDP, the architecture used by QUIC to carry HTTPS traffic [5][6]. Building on UDP lets a transport evolve without waiting for operating-system or network-operator updates [5]. RUDP is a teaching and testing vehicle, not

a production system: because it can be read from end to end, it makes congestion-control behaviour visible, and it provides a platform for studying how a transport reacts when the network misbehaves.

1.3 Purpose and use case

The purpose of the project is a reliable transport that can be read, changed and tested, for the common situation of campus users sending data to a server across a routed network. TCP cannot be modified without changing the operating system, and plain UDP leaves loss, duplication and reordering to the application. A transport built over UDP in user space closes that gap, which is why QUIC and similar protocols are built this way [5][6].

The concrete scenario is a student on the student VLAN uploading data to the server on the server LAN. The access list keeps the student VLAN out of the staff VLAN, and OSPF carries the traffic over the shortest path through R3. If the direct link fails, OSPF reroutes the traffic through R2, which is one hop longer and increases the round-trip time of a running flow. A fixed window rule does not know about this change, while Smart-RUDP observes it through its RTT-ratio state. The network design of Section 9 is therefore not a separate exercise: it supplies the conditions (routing, access policy and failover) under which the transport is meant to run. The protocol itself was tested on one machine with emulated paths (Sections 5 to 8), because Packet Tracer cannot run the C programs; the report states this wherever it matters. To make the protocol visible to a user, a small two-way messenger (RUDP Chat, src/chat.c) was written on the same packet format: each chat message is one RUDP data segment, delivered in order and acknowledged cumulatively, with simulated loss applied on the sending side. In a test with 30% loss in both directions, all 15 messages of one user and all 12 of the other arrived complete and in order, with the lost segments recovered by timeout and fast retransmit.

2. BACKGROUND AND RELATED WORK

Jacobson and Karels (1988) [1] introduced the packet-conservation principle and added several algorithms to 4BSD TCP, among them round-trip-time variance estimation, exponential retransmit backoff, slow start and congestion avoidance. **Chiu and Jain (1989)** [2] analysed increase and decrease rules and showed that additive increase with multiplicative decrease converges to an efficient and fair state; this is why RUDP grows its window linearly and cuts it by half. **RFC 5681** [3] standardises slow start, congestion avoidance, fast retransmit and fast recovery.

CUBIC [4] replaces linear window growth with a cubic function of the time since the last congestion event, which scales better on fast, long-distance paths. It was not implemented in this project and remains future work. **QUIC** [5][6] shows that reliable, congestion-controlled transport can be built on UDP at Internet scale. RUDP shares its UDP foundation and its user-space design, but has no encryption or stream multiplexing. **NetEm** [7] is the Linux traffic-

control module that adds delay, loss, duplication and corruption to packets leaving an interface; it was used here to inject real loss on the loopback interface.

The key researcher for this area is **Van Jacobson**, lead author of [1], whose work underlies every congestion-control mechanism used in RUDP. A fuller literature review accompanies this report.

3. SYSTEM DESIGN

3.1 Overview

RUDP has a client and a server. The client connects to the server on 127.0.0.1, UDP port 8888, performs a handshake, splits a message into 40-byte chunks, sends them under window and congestion control, and closes the connection with a FIN. The server reassembles the chunks in order and acknowledges them. All of this runs over plain UDP sockets.

3.2 Packet format

Every packet starts with a 13-byte header followed by an optional payload of up to 512 bytes. The fields are written into the byte buffer one at a time instead of sending the C structure directly, so compiler padding can never change the layout on the wire.

Table 1: RUDP header layout (13 bytes)

Field	Size	Offset	Purpose
seq_num	4 bytes	0	Sequence number of this packet (counts packets, not bytes)
ack_num	4 bytes	4	Next sequence number the sender of this packet expects
flags	1 byte	8	SYN 0x01, ACK 0x02, FIN 0x04, DATA 0x08; flags may be combined
checksum	2 bytes	9	16-bit integrity check over header fields and payload
payload_len	2 bytes	11	Number of payload bytes that follow

The checksum started as a one's-complement byte sum (all header bytes except the checksum, and the payload bytes, added with the carries folded back and inverted). A fuzzer later showed that this cannot detect swapped or compensating bytes (Section 7.8), so it is now CRC-16/CCITT-FALSE (polynomial 0x1021, initial value 0xFFFF) over seq_num, ack_num, flags, payload_len and the payload in wire order. The field is still 16 bits, so the header layout, the Wireshark dissector and all tools are unchanged; the measurements in this report are not affected by the change. A receiver that finds a mismatch drops the packet, and retransmission recovers it.

3.3 Connection management

The client picks a random initial sequence number and sends a SYN. The server replies with a SYN-ACK whose acknowledgment number is the client's number plus one, and the client confirms with an ACK. The client tries up to five times, waiting three seconds for each reply. The server follows a three-state machine: LISTEN, then SYN_RECEIVED after a SYN, then ESTABLISHED after the final ACK. A FIN is acknowledged and returns the server to LISTEN, ready for a new connection. After the fix described in Section 7.1, the server accepts a new SYN in any state.

3.4 Reliable delivery

The client numbers its data packets consecutively. The server accepts only the packet it expects next; it then advances its expected number and returns a cumulative ACK meaning that everything before that number has arrived. An out-of-order or duplicate packet is discarded and the server repeats its last ACK. This is the Go-Back-N strategy. It was the first design and was later replaced as the default by Selective Repeat (Section 8.9): the server now buffers out-of-order packets and reports them in every ACK, and the sender resends only what is missing. Go-Back-N remains available (--gbn, RUDP_GBN, server built with -DRUDP_GBN) for comparison.

The client recovers losses in two ways. If three duplicate ACKs arrive, it assumes the packet at the base of the window was lost and resends only that packet (fast retransmit). If no new ACK arrives within 500 ms, it times out and resends every packet from the base of the window to the last packet sent. A fixed window constant of 4 was used in Phase 4; once congestion control was added in Phase 5, the window became the congestion window and the constant was left unused.

3.5 Congestion control

The client keeps a congestion window (cwnd) and a slow-start threshold (ssthresh), and logs every change to a CSV file. The rules are shown in Table 2.

Table 2: AIMD congestion-control rules

Event	Action
Start	cwnd = 1, ssthresh = 16
New ACK, cwnd below ssthresh (slow start)	cwnd increases by 1 (about doubling every round trip)
New ACK, cwnd at or above ssthresh (congestion avoidance)	cwnd increases by 1/cwnd (about +1 per round trip)
Three duplicate ACKs (fast retransmit)	ssthresh = cwnd / 2, cwnd = ssthresh
Timeout	ssthresh = cwnd / 2, cwnd = 1 (back to slow start)

Event	Action
Limits	cwnd is never below 1 and never above 32

The treatment of fast retransmit is a simplified form of fast recovery: the window is halved and growth resumes in congestion avoidance, without the window inflation described in RFC 5681 [3]. The retransmission timer is a fixed 500 ms rather than an estimate derived from measured round-trip times [1].

3.6 Loss simulation

To test loss on demand, the client accepts a loss percentage as a command-line argument. Before each data packet is sent, including each retransmission, the client draws a random number and silently discards the packet if it falls below the chosen percentage. Handshake and FIN packets and the server's ACKs are never dropped. The random generator is seeded from the clock, so two runs at the same loss level differ.

4. IMPLEMENTATION

4.1 Environment and build

Development used Windows with WSL2 and Ubuntu, the GCC compiler, Make, Git, Valgrind, tc (iproute2), tshark and Wireshark, with Python and matplotlib for graphs. The code is compiled with the warning flags -Wall and -Wextra, debugging symbols and the C11 standard (Figure 1).

```
Harsh@LAPTOP-IRSGPS8S:~/projects/rudp$ make
mkdir -p bin
gcc -Wall -Wextra -g -std=c11 -o bin/server src/server.c src/common.c
gcc -Wall -Wextra -g -std=c11 -o bin/client src/client.c src/common.c
Harsh@LAPTOP-IRSGPS8S:~/projects/rudp$
```

Figure 1: *Successful build with make: gcc -Wall -Wextra -g -std=c11 produces bin/server and bin/client with no warnings*

4.2 Source files

Table 3: Source files (line counts from the delivered source)

File	Lines	Role
src/common.h	59	Constants, flag bits, packet structure, function prototypes
src/common.c	123	Packing and unpacking, checksum, packet printing
src/client.c	301	Handshake, chunking, window and congestion control, retransmission, logging
src/server.c	190	State machine, in-order delivery, cumulative ACKs, reassembly
Total	673	

4.3 Development phases

Table 4: Implementation phases

Phase	Title	Outcome
0	Environment	WSL2, Ubuntu, toolchain and Git repository set up; build verified
1	Raw UDP echo	Baseline client and server exchanging datagrams
2	Packet format	13-byte header with manual pack/unpack; round-trip and corruption tests; Valgrind clean
3	Handshake	Three-way handshake, state machine and FIN teardown
4	Reliable delivery	Go-Back-N, cumulative ACKs, timeout, fast retransmit; byte-perfect reassembly at 40% simulated loss
5	Congestion control	AIMD with slow start, congestion avoidance and cwnd logging
6	Emulation and capture	Real loss with tc/netem on loopback, captured with tshark and Wireshark
7	Benchmark and graphs	Seven-level loss sweep, CSV output, cwnd and benchmark plots, team handoff

4.4 Version control

The work is stored at github.com/Capsturrrr/rudp. The Phase 7 commit (d9bf3f3, "Phase 7 complete: benchmarking, visualization, resilient server, updated README for team handoff") changed 14 files with 458 insertions and 49 deletions and was pushed to the main branch (Figure 2). No credential is visible in the screenshot; the password prompt was answered at the terminal.

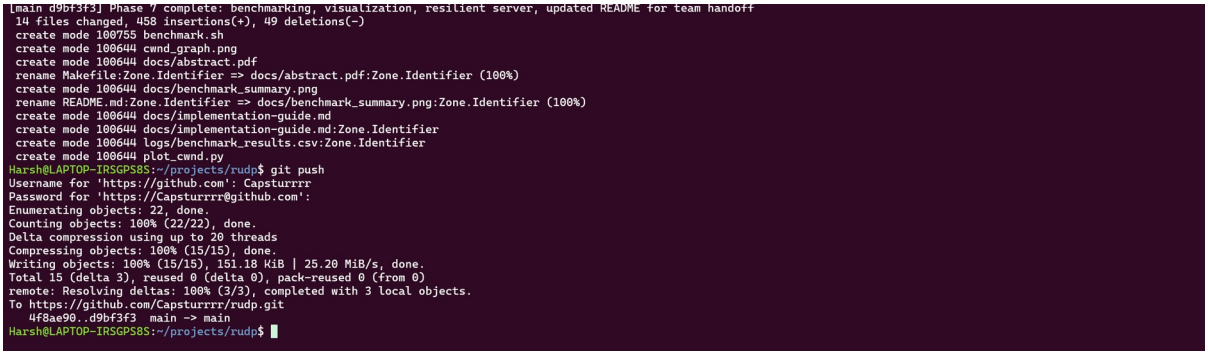


Figure 2: *Phase 7 commit and push to github.com/Capsturrrr/rudp (4f8ae90..d9bf3f3, main -> main)*

5. TEST METHODOLOGY

Three complementary methods were used.

- **Built-in loss simulator.** Drops outgoing client data packets at a chosen percentage (Section 3.6). Used for the benchmark sweep because it is repeatable from a single command.
- **tc/netem on the loopback interface.** The command tc qdisc add dev lo root netem loss 20% makes the kernel itself drop 20% of packets. This checks the protocol against loss that the program did not generate. The rule was always checked with tc qdisc show dev lo and removed afterwards (Section 7.2).
- **Packet capture.** tshark captured the loopback interface inside WSL and the file was opened in Wireshark on Windows, as independent evidence of what crossed the interface.

The benchmark script (benchmark.sh) starts the client once for each loss level of 0, 5, 10, 15, 20, 30 and 40 percent with a 30-second limit, measures wall-clock duration with date and awk, extracts the delivered-packet count and final congestion window from the client output, and writes one CSV row per level. The server must already be running. Each level was run once.

6. RESULTS

6.1 Benchmark sweep

Table 5 and Figures 3 and 4 show the final benchmark. Every level delivered all 62 packets, which together carry the 2,442-byte test message. Transfer time grew from 108 ms with no loss to 20,236 ms at 40% loss, about 187 times longer, while the final congestion window fell from 18.71 packets to 2.00. The terminal output that produced these numbers is shown in Figure 5.

Table 5: Benchmark results, one run per loss level

Simulated loss	Duration (ms)	Packets delivered	Final cwnd
0%	108	62 / 62	18.71
5%	608	62 / 62	7.30
10%	3,112	62 / 62	3.83
15%	7,118	62 / 62	2.00
20%	7,617	62 / 62	2.90
30%	13,625	62 / 62	2.00
40%	20,236	62 / 62	2.00

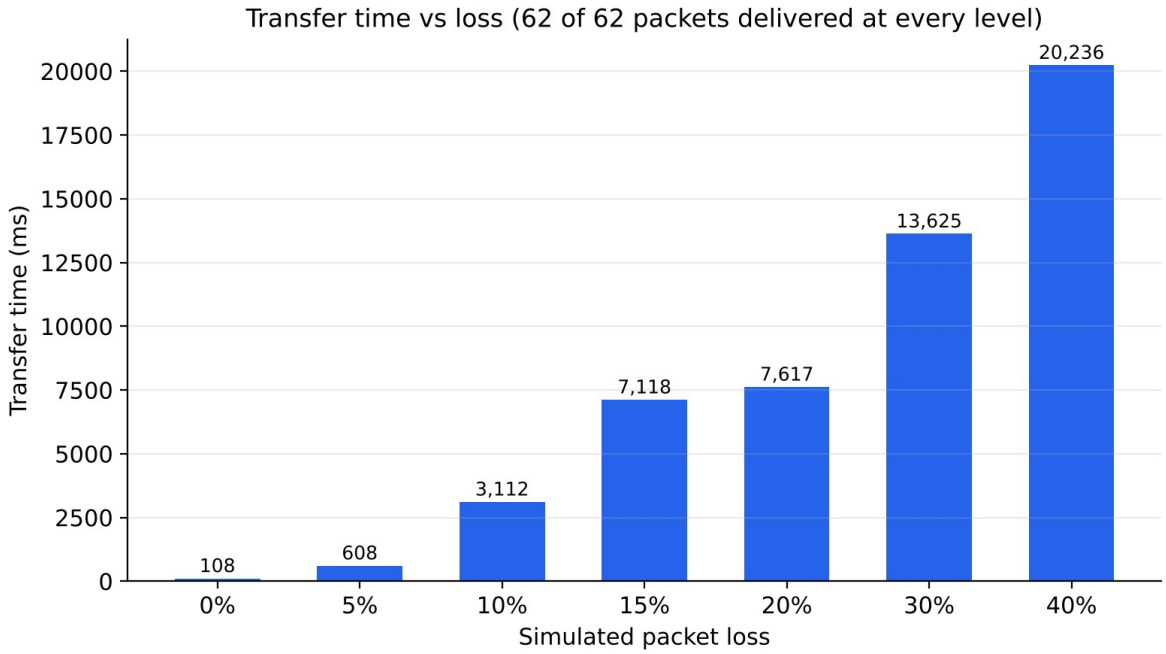


Figure 3: *Transfer time against simulated loss, from the benchmark in Table 5; all 62 packets were delivered at every level*

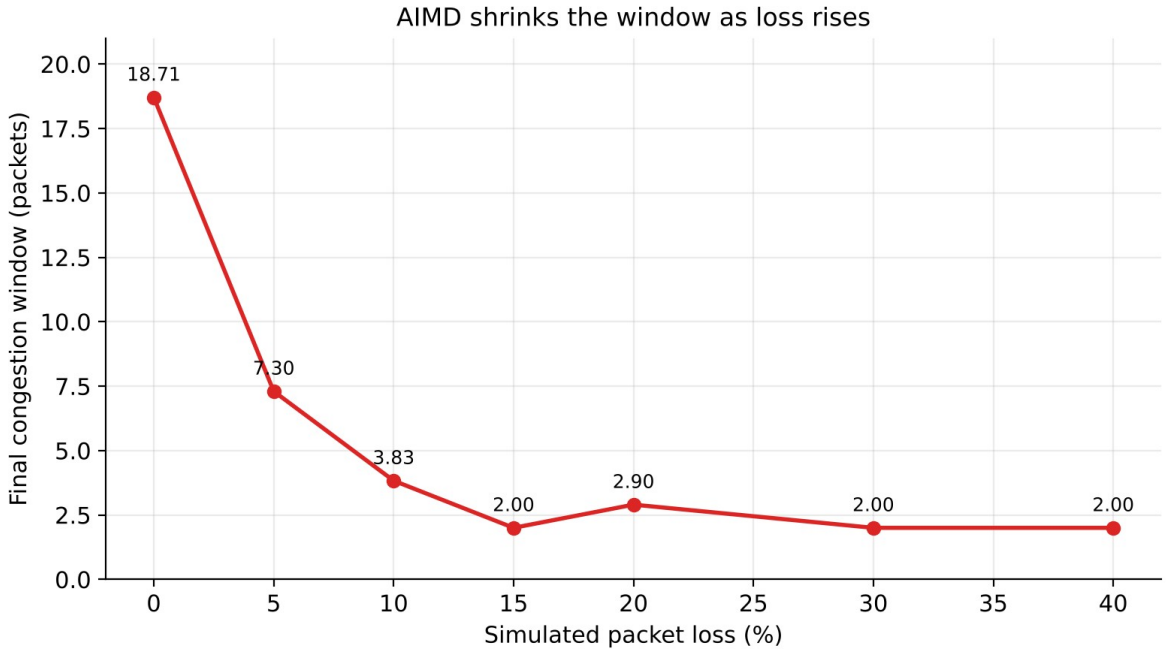


Figure 4: *Final congestion window against simulated loss, from the benchmark in Table 5*

```
=== Testing with 0% loss ===
  Duration: 108ms, Packets: 62, Final cwnd: 18.71
=== Testing with 5% loss ===
  Duration: 608ms, Packets: 62, Final cwnd: 7.30
=== Testing with 10% loss ===
  Duration: 3112ms, Packets: 62, Final cwnd: 3.83
=== Testing with 15% loss ===
  Duration: 7118ms, Packets: 62, Final cwnd: 2.00
=== Testing with 20% loss ===
  Duration: 7617ms, Packets: 62, Final cwnd: 2.90
=== Testing with 30% loss ===
  Duration: 13625ms, Packets: 62, Final cwnd: 2.00
=== Testing with 40% loss ===
  Duration: 20236ms, Packets: 62, Final cwnd: 2.00

Benchmark complete. Results in logs/benchmark_results.csv
loss_percent,duration_ms,packets_sent,final_cwnd
0,108,62,18.71
5,608,62,7.30
10,3112,62,3.83
15,7118,62,2.00
20,7617,62,2.90
30,13625,62,2.00
40,20236,62,2.00
Harsh@LAPTOP-IRSGPS8S:~/projects/rudp$
```

Figure 5: *Terminal output of the final benchmark sweep, with the resulting CSV file*

The 20% result (7,617 ms, final cwnd 2.90) is only slightly slower than the 15% result and ends with a larger window. Because each level was run once with a clock-seeded random generator, such small non-monotonic differences are expected run-to-run variation rather than a protocol effect. The trend across the whole range is clear: more loss means longer transfers and a smaller window.

6.2 Congestion window behaviour

Figure 6 shows the congestion window in a run with no loss. The window starts at 1 and grows by 1 per acknowledgment (slow start) until it reaches the threshold of 16, then grows by about 1/cwnd per acknowledgment (congestion avoidance), ending at 18.71 after 62 packets. The x-axis is the acknowledgment number because the whole transfer takes about a millisecond on the loopback interface.

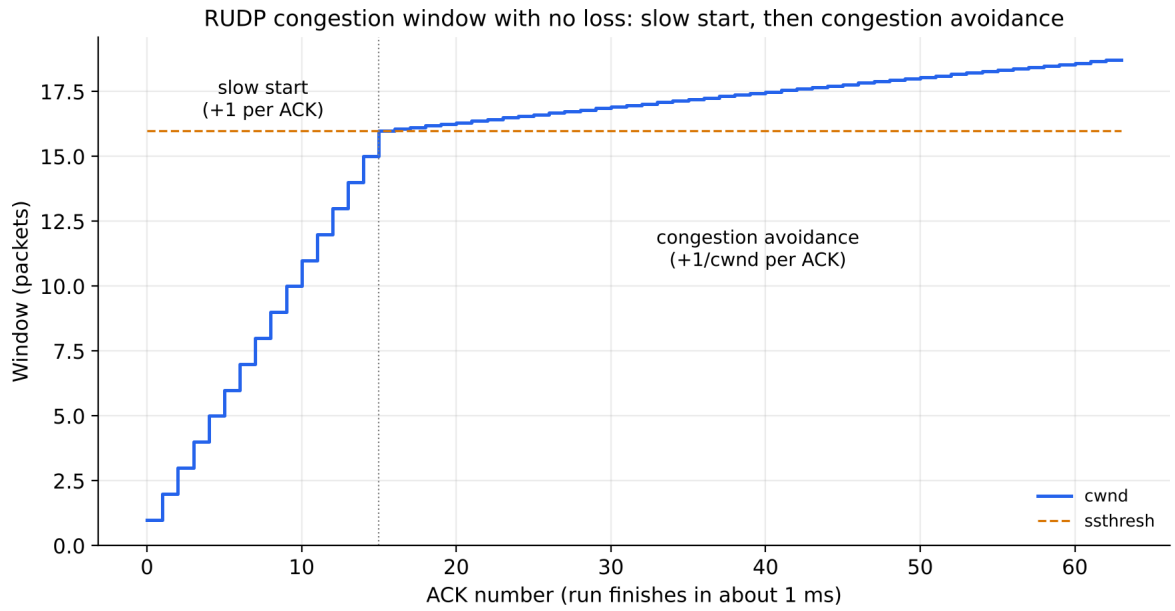


Figure 6: *Congestion window in a run with no loss: slow start up to the threshold of 16, then congestion avoidance*

Figure 7 shows separate runs at 0%, 10%, 20% and 40% simulated loss, plotted from the client's own cwnd log. Each timeout (red dot) resets the window to 1 and halves the threshold; each fast retransmit (green triangle) halves the window. Timeouts occur about 500 ms apart, matching the retransmission timer, and after each reset the window grows again in slow start. This sawtooth is the characteristic signature of AIMD. In these particular runs there were 5 timeouts and 2 fast retransmits at 10% loss, 8 and 6 at 20%, and 45 and 2 at 40%. The plots come from one extra run per level made on 8 October 2026 for these graphs, so their event counts differ from run to run and from the benchmark in Table 5.

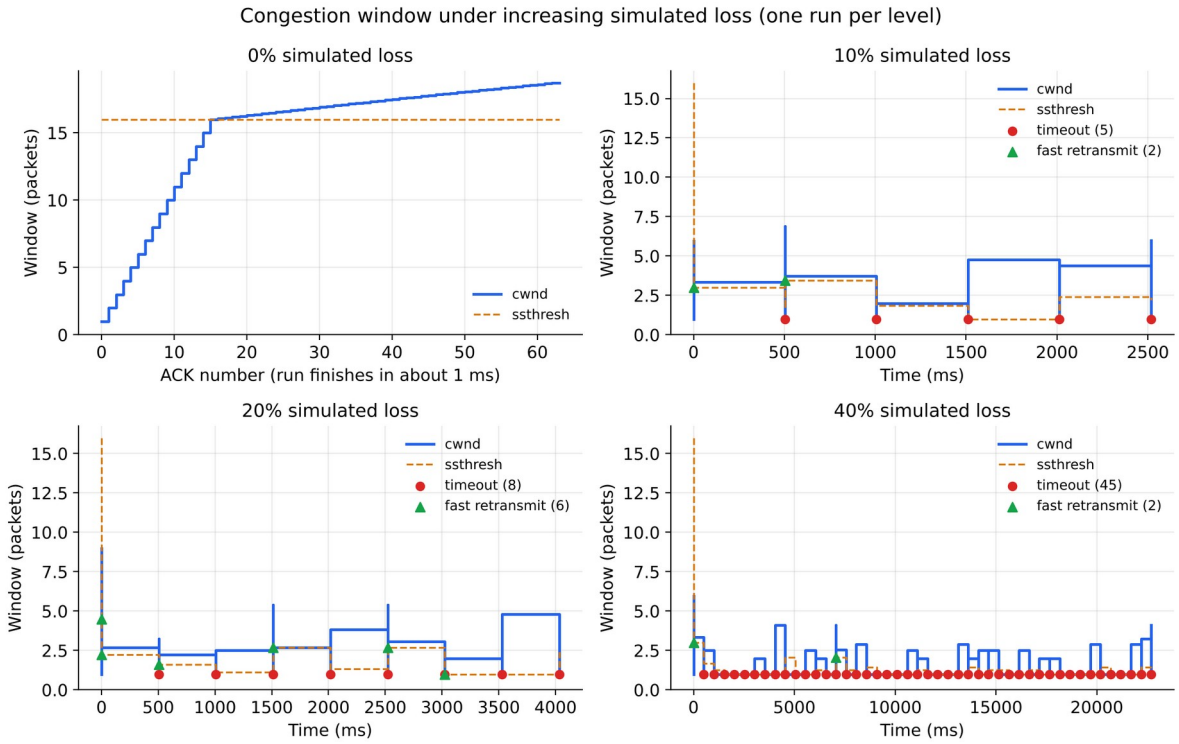


Figure 7: *Congestion window and threshold at 0%, 10%, 20% and 40% simulated loss, with timeouts (red dots) and fast retransmits (green triangles); the 0% panel is plotted against acknowledgment number*

6.3 Real loss and packet capture

With tc/netem dropping 20% of loopback packets, the transfer still completed. In the Wireshark/tshark capture, handshake and acknowledgment packets carried 13 bytes (header only) and data packets carried 53 bytes (13-byte header plus 40-byte payload), consistent with the 40-byte chunk size. Retransmission bursts appeared about 500 ms apart, matching the retransmission timeout, and the client logged 12 timeouts in that run. The capture screenshots are not reproduced in this report; the capture file and logs are kept with the project.

7. PROBLEMS ENCOUNTERED AND HOW THEY WERE FIXED

Every real obstacle is recorded here, with the evidence that was collected at the time.

7.1 Server stuck after an abandoned client

Symptom. The first complete benchmark sweep failed identically at every loss level. Each run took between 15.3 and 15.6 seconds and reported no packets and no window size (Figures 8 and 9). The CSV file contained 0 for both fields in every row.

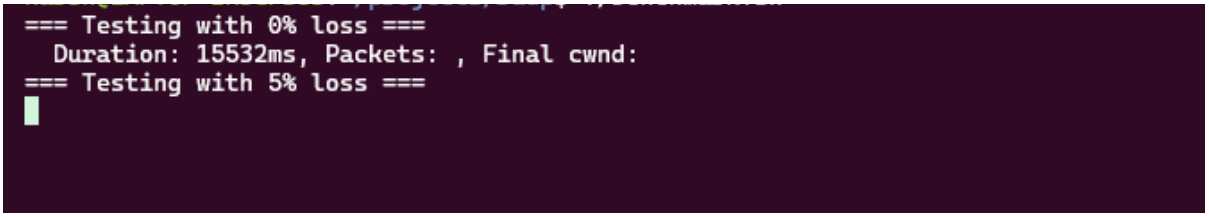


Figure 8: *First symptom: a 15,532 ms run with empty packet-count and window fields*

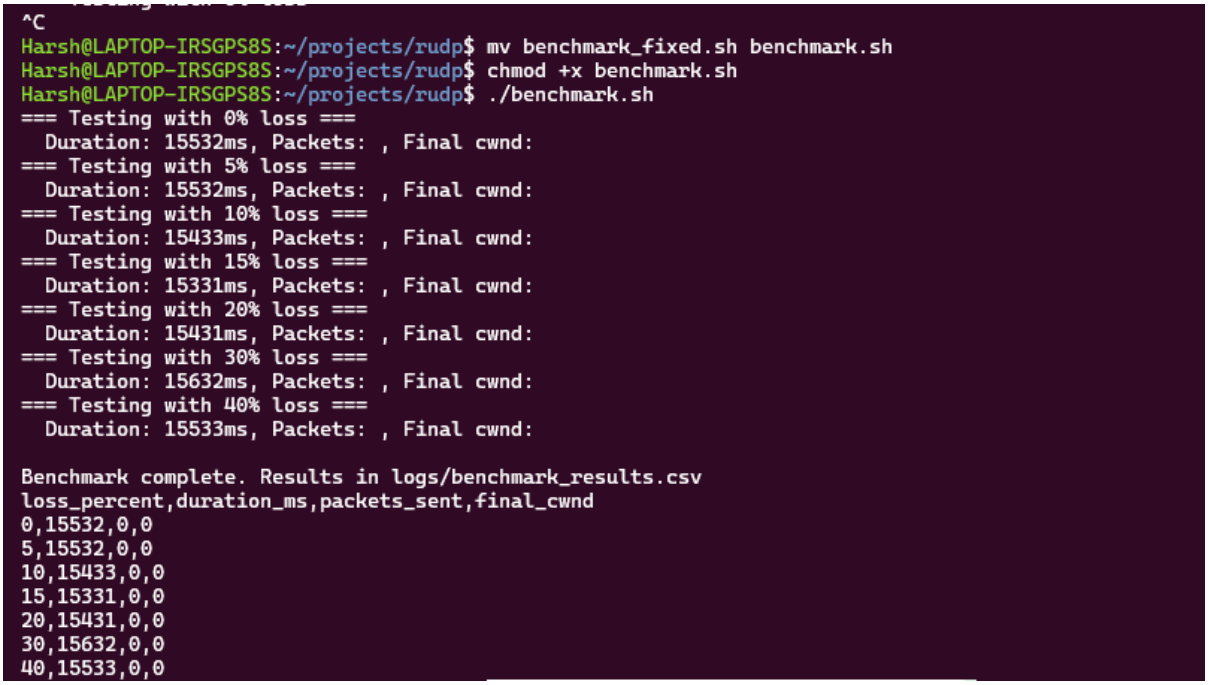


Figure 9: *Whole sweep failing the same way: every level takes about 15.5 s and the CSV records 0 packets and 0 cwnd*

Diagnosis. The command ss -ulnp | grep 8888 showed a server process still bound to UDP port 8888 (Figure 10), so the server was running but not completing handshakes. The duration matched the client's retry limit: five SYN attempts, each waiting up to three seconds, is 15 seconds, plus start-up overhead. The server only accepted a new SYN while in the LISTEN state. An earlier client had been stopped mid-transfer, which left the server in another state where it ignored every later SYN.



Figure 10: *Diagnosis: a server process is still bound to UDP port 8888*

Fix. The server now accepts a new SYN in any state, resets its sequence tracking and reassembly buffer, and replies with a SYN-ACK. This is the condition if (pkt.flags & FLAG_SYN) in src/server.c, replacing a test that also required the LISTEN state.

7.2 A leftover netem rule

Symptom. A test run behaved as though it were hung. **Cause.** A tc/netem rule from an earlier packet-capture session was still active on the loopback interface (Figure 11) and silently

combined with the loss the benchmark was adding itself. **Fix.** The rule was removed with sudo tc qdisc del dev lo root netem, and tc qdisc show dev lo then reported only the default noqueue discipline (Figure 12). From then on the qdisc was checked before every test and deleted immediately after every netem test.

```
Harsh@LAPTOP-IRSGPS8S:~/projects/rudp$ tc qdisc show dev lo
qdisc netem 8001: root refcnt 2 limit 1000 loss 20% seed 4194997339558083968
Harsh@LAPTOP-IRSGPS8S:~/projects/rudp$
```

Figure 11: *Leftover rule found: netem with loss 20% still active on lo*

```
Harsh@LAPTOP-IRSGPS8S:~/projects/rudp$ tc qdisc show dev lo
qdisc netem 8001: root refcnt 2 limit 1000 loss 20% seed 4194997339558083968
Harsh@LAPTOP-IRSGPS8S:~/projects/rudp$ sudo tc qdisc del dev lo root netem
Harsh@LAPTOP-IRSGPS8S:~/projects/rudp$ tc qdisc show dev lo
qdisc noqueue 0: root refcnt 2
Harsh@LAPTOP-IRSGPS8S:~/projects/rudp$
```

Figure 12: *Rule deleted: only the default noqueue qdisc remains*

7.3 Benchmark script produced wrong numbers

Symptom. One run reported a duration of 5,554,676,559 ms, about 64 days, although the packet count (62) and final window (3.55) were correct (Figure 13). **Cause.** The millisecond timestamp command date +%s%3N is not handled correctly by this shell. Separately, an earlier version of the script used grep -oP, which is not portable, and left the packet-count and window fields blank. **Fix.** The script now uses date +%s.%N with an awk calculation for milliseconds, and extracts values with plain grep and sed. The corrected script produced the results in Section 6.

```
=== Testing with 0% loss ===
  Duration: 5554676559ms, Packets: 62, Final cwnd: 3.55
=== Testing with 5% loss ===
```

Figure 13: *Timing bug: an impossible duration of 5,554,676,559 ms next to correct packet and window values*

7.4 Cluttered and misplaced project files

Symptom. The project folder in Windows Explorer showed leftover scripts and a stray server.c in the top-level directory (Figures 14 to 16), together with many files ending in :Zone.Identifier, which Windows adds to files that arrive from elsewhere. **Fix.** server.c was moved into src/, the extra scripts were deleted (Figure 17), and the Zone.Identifier files were removed. The first attempt used the pattern *.Zone.Identifier and matched nothing, because the file names contain a colon (Figure 18); the corrected pattern *:Zone.Identifier worked (Figure 19), leaving a clean directory of Makefile, README.md, benchmark.sh, bin, cwnd_graph.png, docs, logs, plot_cwnd.py, scripts, src and tests.

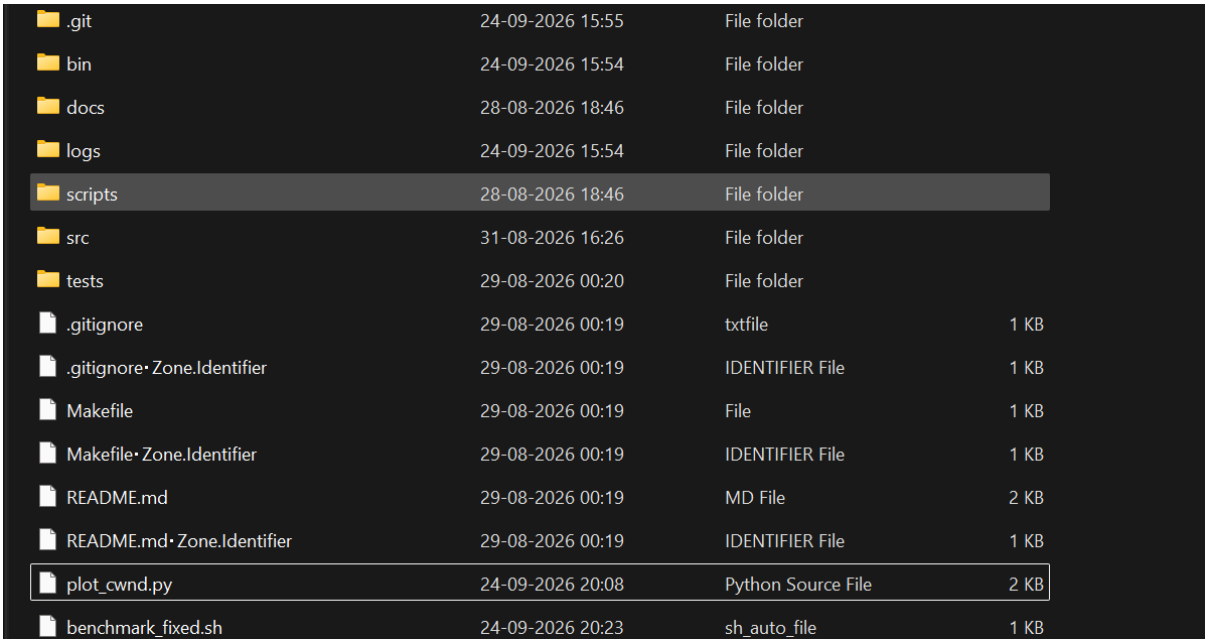


Figure 14: *Project folder in Windows Explorer, including benchmark_fixed.sh*



Figure 15: *Folder listing after file updates; benchmark_fixed.sh still present*



Figure 16: *Stray copies in the top-level directory: benchmark (2).sh and server.c*



Figure 17: *server.c moved into src/ and the extra benchmark scripts removed*



Figure 18: *First clean-up attempt: the wildcard *.Zone.Identifier matched nothing*



Figure 19: *Corrected pattern *:Zone.Identifier leaves a clean directory*

7.5 Wireshark could not see WSL loopback traffic

Capturing on the vEthernet (WSL) interface from the Windows version of Wireshark showed no matching packets, because traffic between two processes inside WSL over 127.0.0.1 never reaches the Windows-visible adapter. The fix was to capture inside WSL with tshark on the lo interface, write a .pcapng file, and copy it to Windows to inspect in Wireshark. The first attempt to write the capture into the home directory failed with permission denied, so it was written to /tmp and copied with elevated rights.

7.6 Wrong assumption about the Windows user name

Copying a capture file to /mnt/c/Users/Harsh/Downloads failed with "No such file or directory" because the Windows account has a different name from the WSL account. Listing /mnt/c/Users showed the real name. The lesson is to confirm a path before using it.

7.7 A credential in a screenshot

A GitHub personal access token appeared in a screenshot shared during setup. A credential that appears in a screenshot or chat should be treated as compromised, so the token was revoked at once and a new one was created and typed only at the terminal prompt. No token appears in any figure in this report.

7.8 A weak checksum, found by a fuzzer

Symptom. A deterministic fuzzer (tests/fuzz_unpack.c, one million iterations under AddressSanitizer and UBSan) flips one to three random bits in valid packets and expects every one to be rejected. About 2% of the corrupted packets were accepted. **Diagnosis.** The checksum was a plain sum of bytes, so any two changes that cancel, or any swap of two bytes, leave it unchanged. **Fix.** CRC-16/CCITT-FALSE in the same 16-bit field. After the fix the fuzzer accepts none of the corrupted packets, no memory errors are reported, and a unit test checks that swapped payload bytes are rejected.

7.9 A lost final handshake ACK hung the connection

Symptom. In the first test over real sockets with 10% loss, two of four runs never finished, while the same scenario in the in-process emulator always did. **Diagnosis.** The emulator never impairs the handshake, so it could not show the problem: if the client's final handshake ACK is lost, the server stays in the half-open state and ignores every data packet that follows, and the client retransmits forever. **Fix.** As in TCP, a DATA packet that arrives while the server is half-open completes the handshake. A regression test (tests/test_handshake.py) drops the final ACK on purpose; it times out without the fix and passes with it.

7.10 Spurious timeouts hid in the baseline

Symptom. The neural agent sent far more retransmissions than AIMD on the fast path (262 against 44), although the path lost only about three packets per run. **Diagnosis.** Tracing a run showed 5 to 15 timeouts. The timer was the smoothed RTT plus four times its deviation; on a path with little jitter the deviation is tiny, and when the agent raises its window by up to 50% at once the last packet of the burst waits behind the others (64 packets at 3,000 packets per second add 21 ms), which is longer than the margin. **Fix.** A floor of 1.3 times the smoothed RTT. On six fast-path runs the agent's retransmissions fell from 206 to 1 and its time from 3.90 s to 3.71 s. The same flaw had been slowing AIMD down (Section 8.11), so all policies now use the floor and the earlier margins were re-measured.

7.11 Two smaller defects in the chat programs

When 100 messages were piped into the terminal chat, it sent only about 20 lines per second: after reading one line, the C library had buffered the rest of the input, so select() saw nothing to read until its 50 ms timeout. Switching standard input to unbuffered mode fixed it. In the web gateway the round-trip time of a packet that had been buffered behind a lost packet was measured when the hole was filled, not when the packet was first reported, which inflated the smoothed RTT to 1.9 s and the timer to its 3 s limit; samples are now taken when a packet is first reported in the bitmap. Both were found by the new gateway-to-terminal interoperability test (tests/chat_interop.sh), which delivers 100 messages each way, in order, at 20% loss on both sides.

8. SMART-RUDP: LEARNING-BASED CONGESTION CONTROL

The AIMD policy used so far is a fixed rule. It halves the window on every loss signal, so it cannot tell a random wireless loss from congestion, and it grows by only one packet per round trip once past the threshold, which is slow on a long satellite path. Recent work applies reinforcement learning to this problem [9][10]. This section extends RUDP into Smart-RUDP by replacing the window-update rule with a Q-learning agent [11], and compares the two under emulated network conditions.

8.1 Design

The agent is a tabular Q-learner implemented in a single header (rl_cc.h, about 70 lines). Time is divided into monitor intervals of about one smoothed round-trip time (at least 10 ms). At the end of each interval the agent observes a state, receives a reward for the previous action, updates its table and chooses a new action. Table 6 lists the design.

Table 6: Smart-RUDP agent design

Element	Definition
State (80)	Mean-RTT ratio bucket (4) x retransmission-fraction bucket (4) x window-size bucket (5). The RTT ratio is the interval mean RTT divided by the minimum RTT plus twice the smallest observed RTT deviation, so natural jitter is not mistaken for queueing.
Action (5)	Multiply the window by 0.5, 0.85, 1.0, 1.15 or 1.5. Increases add at least one packet.
Reward	sqrt(throughput / reference) - 0.5 x max(0, RTT ratio - 1) - 4 x retransmission fraction, in the spirit of the utility functions of PCC [12]. The reference is the bottleneck rate.
Learning	Q(s,a) is updated with learning rate 0.15 and discount 0.9. Exploration probability decays from 0.35 to 0.03 over 40 training episodes. The table starts from a prior that favours growth when the path looks clean and shrinking when it looks congested.

Element	Definition
Safety rule	After a retransmission timeout the window is halved (minimum 2) in Smart-RUDP, and reset to 1 in the AIMD baseline.

Both modes share the same reliability code (Go-Back-N when the experiments of Sections 8.1 to 8.8 were run; Selective Repeat is now the default, Section 8.9): an adaptive retransmission timeout from the smoothed RTT and its deviation (clamped between 60 and 3,000 ms, doubled after a timeout), and fast retransmit on three duplicate ACKs that resends the whole in-flight window. Only the window policy differs. The baseline therefore differs from the original RUDP client of Section 3 in using the adaptive timeout and in a 128-packet window cap; it keeps slow start to a threshold of 16 followed by additive increase.

8.2 Experimental setup

Real tc/netem needs root and was not available in the build environment, so the path is emulated inside the client: random loss, a bottleneck with a fixed service rate and a tail-drop queue, one-way delay with jitter, and a delayed return path for acknowledgments. Packets still travel through real UDP sockets to the real RUDP server. Three scenarios were used, each trained separately. The loss rates are modest because Go-Back-N degrades sharply at high loss (Section 8.4).

Table 7: Emulated scenarios

Scenario	One-way delay	Jitter	Random loss	Rate	Queue
5G-like	10 ms	3 ms	0.2%	1,500 pkt/s	60 pkt
Lossy wireless	25 ms	10 ms	2%	800 pkt/s	50 pkt
Satellite-like	150 ms	10 ms	0.3%	400 pkt/s	100 pkt

For each scenario the agent was trained for 40 episodes (500 packets per episode, 800 for the satellite case), then its table was frozen and a 1,200-packet transfer was run six times against six runs of the AIMD baseline, using the same starting random seeds.

8.3 Results

Table 8: Mean of six 1,200-packet transfers per scenario (time shown with standard deviation)

Scenario	AIMD time (s)	Smart time (s)	AIMD retx	Smart retx	AIMD RTT (ms)	Smart RTT (ms)
5G-like	1.44 +/- 0.21	2.35 +/- 0.04	62	34	25.1	23.8

Scenario	AIMD time (s)	Smart time (s)	AIMD retx	Smart retx	AIMD RTT (ms)	Smart RTT (ms)
Lossy wireless	9.30 +/- 0.89	9.14 +/- 0.79	259	263	59.8	59.5
Satellite-like	22.0 +/- 2.9	24.3 +/- 2.3	105	88	311.3	309.0

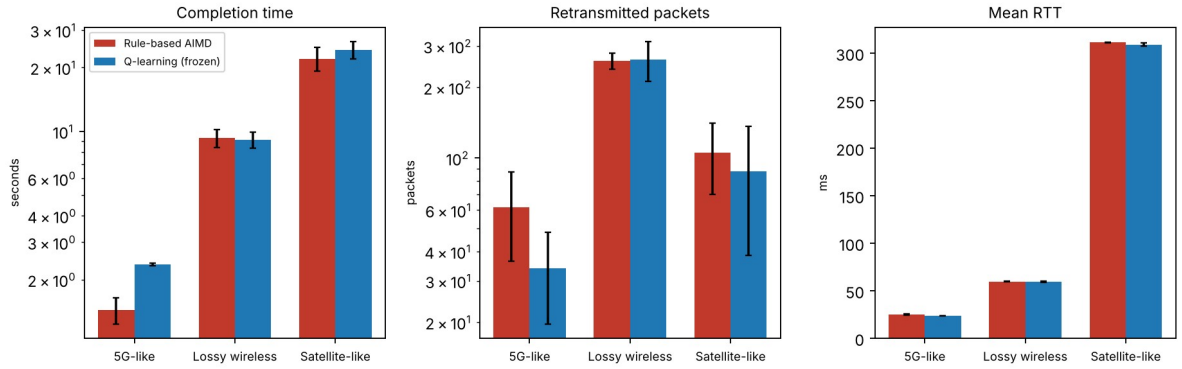


Figure 20: *Completion time, retransmitted packets and mean RTT for the rule-based baseline and the frozen Q-learning agent (mean of six runs, error bars are one standard deviation)*

Smart-RUDP did not beat the baseline on completion time. With six runs it was slower on the 5G-like and satellite-like paths (by about 63% and 10%) and equal within noise on the lossy path; a repeat with 20 runs per mode (Table 8b) gave tighter intervals and confirms the same pattern, with a larger satellite gap. It sent fewer retransmissions on the 5G-like and satellite-like paths and had a slightly lower mean RTT, but the retransmission differences are within one standard deviation across runs and should not be treated as significant. The result is reported as obtained: with a tabular agent, 40 training episodes per scenario and six evaluation runs, learning-based control matched but did not outperform a well-tuned rule on these paths. Section 8.5 reports a neural agent that does outperform it.

Table 8b: Mean of 20 runs per mode and scenario with 95% confidence intervals (frozen Q-tables, 1,200 packets, seeds 2001 to 2020; scripts eval_ci.sh and ci_summary.py)

Scenario	AIMD time (s)	Smart time (s)	Paired difference (s)	Significant?
5G-like	1.36 +/- 0.09	2.29 +/- 0.06	+0.92 +/- 0.12	yes, slower
Lossy wireless	9.27 +/- 0.54	8.90 +/- 0.46	-0.37 +/- 0.45	no (tie)
Satellite-like	19.14 +/- 1.75	25.46 +/- 1.22	+6.31 +/- 2.24	yes, slower

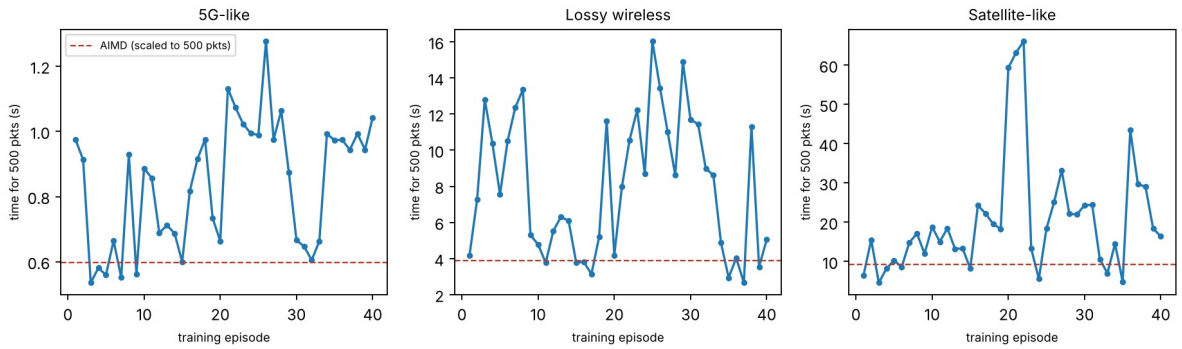


Figure 21: *Training curves: time to deliver 500 packets in each training episode (dashed line: baseline, scaled to the same length)*

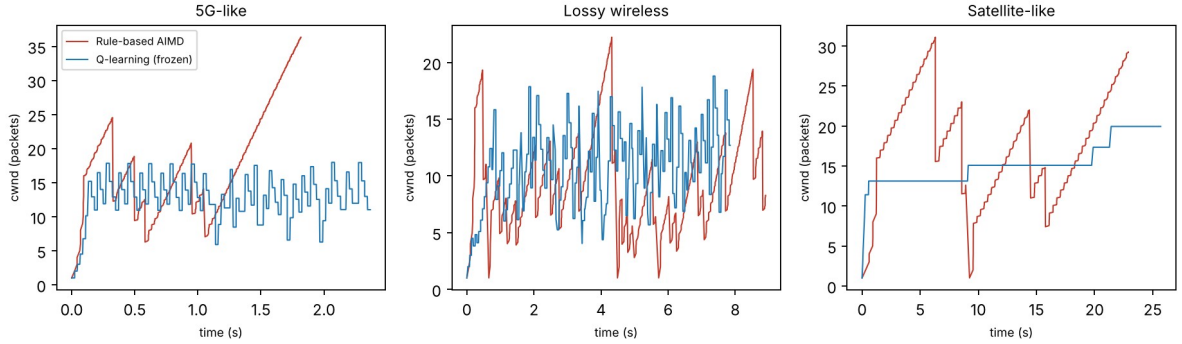


Figure 22: *Congestion window over time in one evaluation run of each mode and scenario*

The training curves in Figure 21 are noisy and do not show a steady improvement; in the 5G-like case later episodes are on average slower than early ones, which suggests that the agent is still exploring and that the training budget is too small for this table. Figure 22 shows that in the 5G-like and satellite-like runs the learned window stays within a narrow band (about 10 to 20 packets) while AIMD climbs to higher peaks and falls back, which is consistent with the learned policy being more conservative and finishing later. In the lossy run the learned window changes rapidly from one interval to the next.

8.4 Development notes and limits

- The first training round was far worse than the baseline (about four times slower on the satellite path). Three changes were made before the reported run: the prior was corrected to encourage growth under light loss, the window is halved rather than reset to one after a timeout, and the throughput term of the reward was square-rooted so that its signal is not drowned by the loss penalty at low utilisation. The reported numbers are from the final configuration and were not selected from several runs of it.
- Go-Back-N limits what any window policy can achieve: with loss of a few percent, most windows contain a loss and the receiver discards everything after it. Selective acknowledgments would help both modes, and Selective Repeat is now the default (Section 8.9).

- The emulated path in the C program keeps packets in order unless the option --reorder 1 is given; reordering was tested in the Python simulator (Section 11) and in one C scenario (Table 8d). A TCP model and plain UDP were compared in simulation; real kernel TCP over an impaired path was not run because the build environment has no tc/netem (a script, netem_baselines.sh, is provided for a Linux machine).
- The scenarios were repeated on kernel tc/netem (Section 8.17); the results agree in direction with the emulated path but not in size (AIMD is much slower on the real satellite path).

8.5 Neural agent, chat-style table agent and selective acknowledgments in the C transport

After Section 11 produced a neural agent and a retrained table in the Python simulator, both were added to the C transport so they could be tested on the same three emulated paths. The C client now has the modes rl2 (the retrained table, with the rule that the window may not shrink without a loss or delay signal, window cap 32 and a decision every 100 to 500 ms) and deep (the neural network of Section 11, a forward pass in src/deep_cc.h with weights exported by web/export_dqn.py). AIMD and the new modes all start from a window of 10 for a fair comparison. The agents were trained only in the Python simulator, on paths that are different from these three, so the C scenarios are unseen. Table 8c gives 15 runs per mode for the 5G-like and lossy paths and 8 runs for the satellite-like path, with 95% confidence intervals.

Table 8c: Completion time of 1,200 packets in the C transport (mean +/- 95% confidence interval; the last two columns are paired differences; all differences shown are significant)

Scenario	AIMD time (s)	rl2 time (s)	Deep time (s)	Deep - AIMD (s)	rl2 - AIMD (s)
5G-like	1.38 +/- 0.16	2.64 +/- 0.50	1.18 +/- 0.03	-0.20 +/- 0.15	+1.25 +/- 0.56
Lossy wireless	9.13 +/- 0.49	7.54 +/- 1.26	6.63 +/- 1.45	-2.49 +/- 1.42	-1.58 +/- 1.30
Satellite-like	17.93 +/- 2.68	38.07 +/- 0.33	13.12 +/- 0.45	-4.30 +/- 2.51	+20.65 +/- 2.56

The neural agent finished faster than AIMD on all three paths, by 0.20 s (15%) on the 5G-like path, 2.49 s (27%) on the lossy path and 4.30 s (24%) on the satellite-like path. This is the first controller in this project that beats AIMD in the C experiments. The retrained table did not carry over: it was faster on the lossy path but 91% slower on the 5G-like path and more than twice as slow on the satellite-like path, where its window stayed small. The neural network, trained on the same paths with the same reward, generalised to these unseen paths and the table did not, which is the argument for a function approximator over a table. The caveats are the same as

before: the agents were chosen on simulated validation paths, the paths here are emulated inside the client, AIMD keeps its original cap of 128 while the agents are capped at 32 (Section 8.7 repeats the comparison with equal caps and the conclusion holds), and the neural agent retransmitted more packets than AIMD on the lossy path (about 600 against 250), so it wins by sending more aggressively into a path whose loss is random.

Selective acknowledgments were also added to the C transport (a server built with -DRUDP_SACK buffers out-of-order packets and returns an 8-byte bitmap in every ACK; the client option --sack 1 then resends only the holes), together with a --reorder option that lets jitter reorder packets on the emulated path. Table 8d compares the Go-Back-N client on the plain server with the selective client on the selective server, 8 runs per cell. Selective acknowledgment cut retransmissions by about ten times in every scenario (for AIMD from 232 to 23 on the lossy path and from 245 to 22 with reordering). It did not change completion time significantly in these emulated scenarios, where the bottleneck rate limits throughput and extra retransmissions cost little; the large time gains seen in the simulator (Section 11) did not carry over. One interaction is worth knowing: a Go-Back-N client talking to the selective server was slower than on the plain server (13.2 s against 8.1 s on the lossy path), because the cumulative ACK jumps when a hole is filled, so the two sides should be used as a pair.

Table 8d: Mean completion time (s) of 1,200 packets with Go-Back-N and selective acknowledgment in the C transport (8 runs each, seeds 4001 to 4008; only the 5G-like and satellite-like differences are very small; none of the time differences is significant at 95%)

Scenario	AIMD GBN time	AIMD SACK time	Deep GBN time	Deep SACK time	Retx AIMD GBN / SACK
5G-like	1.26	1.26	1.17	1.18	55 / 2
Lossy wireless	8.07	8.50	8.72	4.56	232 / 23
Lossy, reordering	10.22	9.24	6.99	5.09	245 / 22
Satellite-like	18.88	18.11	13.28	13.32	87 / 4

8.6 Fairness, ablation study and chat gateway upgrades

Fairness. Two flows were run through one shared bottleneck in a multi-flow version of the simulator (300 packets per flow, 20 seeds). Table 8e reports flow A's share of the packets delivered when the first flow finished and the Jain fairness index (1.0 is a perfectly equal split; 0.5 is the worst case for two flows). The neural agent shares fairly with another neural agent (index 0.92 to 0.98) and with AIMD on the short paths (0.95 to 0.99). On the long 120 ms path it takes 74% of the bottleneck against an AIMD flow (index 0.82), which is about as unfair as a fixed window of 8 (72%, index 0.84). It is not a starvation case, since the AIMD flow still

finishes, but the agent is not a polite sender on that path. A second finding is that two neural flows on a congestion-only path each finish more slowly (18.9 s) than two AIMD flows (12.2 s), so the single-flow gain does not simply scale to shared links. Fairness was not tested in the C transport.

Table 8e: Share of flow A (first-named controller) and Jain index for two flows on a shared bottleneck (simulator, 20 seeds)

Path	aimd/ aimd	deep/ deep	deep/ aimd	smart/ aimd	fixed/ aimd	Metric
Congestion only	52 / 0.998	56 / 0.984	44 / 0.987	38 / 0.948	29 / 0.845	share % / Jain
Congestion + 2% loss	50 / 0.990	55 / 0.921	60 / 0.952	54 / 0.965	62 / 0.944	share % / Jain
Long path (120 ms)	51 / 0.999	59 / 0.969	74 / 0.819	61 / 0.953	72 / 0.843	share % / Jain

Ablation study. To find out which parts of the neural agent matter, it was retrained with one design choice removed (4,000 episodes, four seeds each, best seed chosen on the validation paths, scored on the 48 stress paths; Table 8f). The only component whose removal clearly hurts is the no-shrink mask: without it the best network averaged 6.54 s, close to AIMD (7.16 s), against 3.2 to 3.6 s with it. Removing the RTT-delay penalty, the last-action input or the throughput input did not hurt in the best seed (3.2 to 3.4 s); with the shorter 4,000-episode training the full agent was not better than those variants (3.59 s). The larger effect is the seed: the same recipe gave between 3.2 s and 7.4 s depending on the random seed, which is why the final agent was chosen on validation paths. The study therefore supports the no-shrink mask as the key ingredient and does not show a measurable value for the other three, and the seed sensitivity is a real weakness of the method.

Table 8f: Mean completion time on 48 unseen stress paths for each ablation (simulator; AIMD 7.16 s on the same paths)

Variant	Best seed (s)	All four seeds (s)
Full agent	3.59	5.21, 7.10, 3.59, 7.31
No no-shrink mask	6.54	6.60, 7.44, 7.24, 6.54
No RTT-delay penalty	3.35	3.33, 3.24, 7.33, 3.35
No last-action input	3.23	6.68, 7.45, 7.15, 3.23
No throughput input	3.41	3.31, 3.41, 3.36, 3.24

Chat gateway upgrades. The web chat gateway gained a live dashboard (window, packets in flight, RTT and loss over the last 60 seconds, drawn from the gateway's own samples), an

optional RFC 6298 adaptive retransmission timer with exponential backoff (--rto adaptive), and a fix for a retransmission storm. The storm was found while testing the dashboard: with delayed links, duplicate ACKs from an earlier fast retransmit kept arriving after the window had been resent, and each batch of three triggered another full resend, giving about 200,000 retransmissions in one 400-message run at 10% loss. The gateway and the terminal chat now allow a new fast retransmit only after the data outstanding at the last one has been acknowledged, as the C smart client already did. With the fix, a 200-message run sent 385 to 1,352 packets depending on the controller. On a 600 ms round-trip path with 5% loss, the adaptive timer finished 100 messages in 17.8 s with 54 retransmissions, against 29.0 s and 201 retransmissions for the fixed 300 ms timer, which is shorter than the round trip and times out spuriously (one run each, so illustrative only).

8.7 Validity check: equal window caps and a more realistic simulator

Two weaknesses in the comparison were found and tested. **First, the window cap.** AIMD in the C transport was capped at 128 packets and the agents at 32. Table 8g repeats the C experiment with AIMD capped at 32 as well (10 runs per cell, 95% intervals, paired by seed). The original neural agent still beats AIMD at the equal cap on all three paths: by 0.28 s (19%) on the 5G-like path, 2.61 s (29%) on the lossy path and 5.37 s (28%) on the satellite-like path, all significant. Against the old cap of 128 the 5G-like difference is not significant, and the other two are. The cap therefore does not explain the result.

Table 8g: Mean completion time (s) of 1,200 packets in the C transport with equal window caps (seeds 5001 to 5010; the retrained agent is the one described below)

Scenario	AIMD cap 128	AIMD cap 32	Neural (original)	Neural (retrained)	Runs
5G-like	1.31 +/- 0.19	1.48 +/- 0.17	1.20 +/- 0.04	2.88 +/- 0.03	10
Lossy wireless	8.89 +/- 0.58	9.12 +/- 1.18	6.51 +/- 0.93	9.53 +/- 0.49	10
Satellite-like	18.66 +/- 2.73	18.85 +/- 1.92	13.48 +/- 0.40	30.02 +/- 6.42	10

Second, the simulator. The Python simulator that trained the agent and produced the stress results allowed a duplicate-ACK retransmission storm, the same flaw fixed in the chat gateway (Section 8.6): every duplicate ACK batch could trigger another full resend. At high loss this acts as aggressive redundancy and flatters aggressive controllers. With the realistic rule switched on (RUDP_RG=1: no new fast retransmit until the data outstanding at the last one is acknowledged, as in the C smart client), the stress result changes a great deal. Mean delivery time on the 48 stress paths was 7.27 s for the original neural agent, 7.10 s for the table agent, 7.16 s for the fixed window and 7.75 s for AIMD; the neural agent was faster than AIMD on 12 paths, slower on 15 and within 5% on 21. Training a new neural agent in the realistic simulator

gave a stable result (all four seeds within 4.84 to 4.93 s on validation, against the earlier spread of 3.2 to 7.4 s) and a stress mean of 7.03 s, which is again about level with AIMD (faster on 13, slower on 13). That agent was much worse in the C transport (Table 8g), so it was not adopted. The honest conclusion is that the large simulated gains reported in Section 11 (3.20 s against 7.16 s) came partly from the storm effect and should not be quoted as general; the defensible evidence for the neural agent is the C measurement in Table 8g, which uses the realistic rule, an equal cap and paired runs, and it is limited to three emulated paths.

One more idea was tested and dropped: treating loss without RTT growth as random and refusing to shrink the window (a guard in web/cc.py, RANDOM_LOSS_RATIO). On 60 validation paths it changed the mean time from 3.83 s to between 3.87 s and 3.88 s, so it gave no gain and is off by default.

8.8 Hand-over to AIMD under heavy loss (hybrid agent)

The validity check exposed a serious weakness: at high random loss the neural agent is far worse than AIMD. A new C scenario with 10% loss (40 ms delay, 600 packets per second, queue 40, 300 packets) took the neural agent 85 s on average against 11 s for AIMD capped at 32, with about 1,300 packets sent for 300 delivered. The agent was trained mostly on mild loss and keeps its window large when the path is lossy. Two repairs were tried in the C transport. Cutting the window to a quarter on every retransmission timeout (--tcut 0.25) helped the 10% scenario (19 s) but made the 2% loss path much worse (24 s against 9 s), so it was dropped. A hand-over rule (--hybrid) works better: while the fraction of lost packets in the last decision interval is at least a threshold (5% or 10%), AIMD controls the window (reset to 1 on a timeout, halve on duplicate ACKs, additive growth), and after three clean intervals the agent takes over again.

Table 8h: Mean completion time (s) in the C transport, 95% intervals (seeds 7001 onward; the 10% loss scenario sends 300 packets, the others 1,200)

Scenario	AIMD cap 32	Neural	Hybrid 5%	Hybrid 10%	Runs
5G-like	1.48 +/- 0.21	1.17 +/- 0.03	1.19 +/- 0.04	1.23 +/- 0.07	10
Lossy wireless (2%)	9.04 +/- 0.79	7.53 +/- 1.91	7.21 +/- 1.41	7.57 +/- 0.80	10
Satellite-like	19.68 +/- 2.99	13.39 +/- 0.29	13.70 +/- 0.45	13.70 +/- 0.32	10
Hostile, 10% loss (300 packets)	11.08 +/- 2.78	84.85 +/- 32.00	17.57 +/- 10.58	13.91 +/- 6.50	5

With the hand-over, the three benign paths keep the neural agent's advantage over AIMD (significant for all three with either threshold; for the lossy path the plain agent's difference of 1.5 s was not significant in this run, while the hybrid's was), and the hostile path drops from 85 s to 14 to 18 s, which is not significantly different from AIMD (paired difference +2.8 s with an interval of 8.3 s for the 10% threshold). The hybrid agent is therefore the recommended configuration (--hybrid 0.10). In the calibrated simulator (Section 8.7, in-order jitter, realistic recovery and adaptive timeout; 70 random paths of 300 packets) the plain agent averaged 73.3 s against 49.3 s for AIMD, and the hybrid 53.0 s (faster than AIMD on 25 paths, slower on 31): it removes most of the damage and gains only on paths with 1 to 8% loss (28.4 s against 31.0 s). The honest summary is that the learned controller helps on clean-to-moderate paths and needs a classical controller as a safety net everywhere else. The chat gateway has the same options (--hybrid, --tcut), and the test suite covers the hand-over.

8.9 Selective Repeat replaces Go-Back-N as the default

Go-Back-N was the weakest part of the design, so it has been replaced as the default in the C transport, the terminal chat and the web chat gateway. The receiver (server.c, chat.c, rudp_web.py) buffers out-of-order packets within a 64-packet window instead of discarding them and returns, in every ACK, the cumulative number plus an 8-byte bitmap of the packets it already holds. The sender (client.c, smart_client.c, the chat programs) keeps one timer per packet and resends only the packets that have expired and are not reported, and a fast retransmit resends only the holes below the highest reported packet. The window is capped at 60 so it fits the receiver's buffer. This is Selective Repeat with a cumulative ACK and a bitmap rather than one ACK per packet. Go-Back-N is still available for comparison (--gbn for smart_client, RUDP_GBN=1 for the original client and the terminal chat, --gbn for the gateway, and a server built with -DRUDP_GBN), and all tables in Sections 8.1 to 8.8 were measured with Go-Back-N and are labelled as such. The test suite now covers both modes (end to end at 20% loss for the original client in both modes, and all transport modes against both receivers).

Table 8i repeats the main C comparison with Selective Repeat on both sides (10 runs per cell, 95% intervals, paired by seed, AIMD capped at 32). The neural agent now beats AIMD by 9% on the 5G-like path (0.11 s), 46% on the lossy path (3.7 s) and 50% on the satellite-like path (12.8 s), all significant, a larger margin than with Go-Back-N. The hostile 10% loss path, where the plain neural agent took 85 s with Go-Back-N, now takes 8.1 s against 7.3 s for AIMD, which is not significantly different, with 329 packets sent for 300 delivered; the hand-over of Section 8.8 neither helps nor hurts (6.9 s). The large failure of Section 8.8 was therefore mainly a Go-Back-N effect, since a lost packet no longer causes the whole window to be resent. One result goes the other way: AIMD itself was slower on the satellite-like path with Selective Repeat (25.7 s) than with Go-Back-N (19.7 s in Table 8h); the cause was not investigated, and the neural agent is faster than AIMD under both.

Table 8i: Mean completion time (s) with Selective Repeat in the C transport (seeds 8001 onward)

Scenario	AIMD cap 32	Neural	Hybrid 10%	Neural - AIMD	Runs
5G-like	1.27 +/- 0.10	1.16 +/- 0.02	1.16 +/- 0.02	-0.11 s	10
Lossy wireless (2%)	7.99 +/- 0.70	4.29 +/- 0.29	4.29 +/- 0.30	-3.69 s	10
Satellite-like	25.70 +/- 2.82	12.91 +/- 0.26	13.00 +/- 0.27	-12.78 s	10
Hostile, 10% loss (300 packets)	7.34 +/- 1.96	8.07 +/- 6.97	6.90 +/- 4.09	+0.74 s (n.s.)	5

The other implementations show the same direction. In the web gateway with 10% loss in each direction and 50 ms delay, 200 messages took 6.4 s with AIMD and Selective Repeat (23 retransmissions) against 11.9 s (141 retransmissions) with Go-Back-N, and 2.7 s against 8.6 s for the neural agent (37 against 877 retransmissions); every message arrived complete and in order. In the terminal chat at 30% loss in each direction, 30 messages were delivered with 53 packets sent (23 retransmissions) against 132 (102). These are single runs and illustrate the effect rather than measure it. A Go-Back-N sender talking to a Selective Repeat receiver still works but gets none of the benefit, and the reverse pairing delivers correctly but resends each missing packet only when its own timer expires.

8.10 Performance tuning: larger windows and the new defaults

The performance pass started from two measured limits. First, the window cap: the satellite-like path (150 ms each way, 400 packets per second) can hold about 120 packets in flight, but the agents and the AIMD baseline were capped at 32, so throughput could not exceed about 107 packets per second. Second, the AIMD baseline itself: it starts additive growth at a window of 16, which at 300 ms round trip takes about 22 round trips to reach 32. To remove the first limit the Selective Repeat receiver window was raised from 64 to 256 packets (a 32-byte bitmap in every ACK) and the sender cap to 250. To keep the comparison honest the baseline was tuned as well: AIMD with a 128 cap and an initial threshold of 128 (aimd_128) is reported next to the original (cap 32, threshold 16). CUBIC (cap 128) was also implemented and tested (--mode cubic): it beat tuned AIMD on the satellite-like and fast paths (16.7 s against 19.1 s and 14.4 s against 17.7 s) but lost on the 5G-like path and was twice as slow on the 2% loss path (17.5 s against 8.1 s), and the neural agent was faster than CUBIC on every path in that screening. CUBIC is evaluated again, with the corrected retransmission timer, in Section 8.11.

Table 8j reports the screening runs (5 runs per cell, seeds 9001 to 9005) and Table 8k the final comparison (10 runs, 5 for the hostile path). The neural agent was not retrained for larger windows: its inputs still divide the window by 32, so it sees values outside its training range, yet

it generalised. Raising its cap from 32 to 64 cut the satellite-like time from 13.1 s to 8.2 s, and 128 to 6.3 s; 250 gave no further gain on that path (6.9 s) and was slightly faster only on the fast path (3.7 s against 4.0 s) at the cost of more retransmissions. The new default is therefore a cap of 128 together with the AIMD hand-over of Section 8.8 (--hybrid 0.10). The older settings remain available (--maxcwnd 32 --hybrid 0, or RUDP_CLASSIC=1, which the earlier evaluation scripts now set).

Table 8k: Mean completion time (s) with the final configuration (Selective Repeat; seeds 9701 onward; n.s. = not significant at 95%)

Path (10 runs)	AIMD, cap 32	AIMD, cap 128	Neural, new default	Gain over cap-128 AIMD
5G-like, 1,200 packets	1.30 +/- 0.12	1.09 +/- 0.13	1.09 +/- 0.05	0% (n.s.)
Lossy 2%, 1,200 packets	8.44 +/- 0.72	8.27 +/- 0.54	4.52 +/- 0.55	45%
Satellite-like, 1,200 packets	25.13 +/- 2.94	20.04 +/- 4.19	6.32 +/- 0.42	68%
Fast, high-delay, 3,000 packets	19.79 +/- 1.08	17.29 +/- 1.43	4.01 +/- 0.22	77%
Hostile 10% loss, 300 packets (5 runs)	7.53 +/- 1.10	8.51 +/- 1.70	5.89 +/- 2.50	31% (n.s.)

The fast path (60 ms each way, 3,000 packets per second, queue 300) is new in this section. The new default finishes 3,000 packets in 4.0 s (750 packets per second) against 17.3 s for tuned AIMD; the paired differences against both baselines are significant on the lossy, satellite-like and fast paths (45 to 80% faster), equal on the 5G-like path (where the transfer is short) and not significant on the hostile path. It sent more retransmissions than AIMD on the fast path (262 against 44 for 3,000 packets). Section 8.11 shows that these were spurious timeouts caused by a retransmission timer that was too tight for the agent's window bursts, and that fixing the timer also sped up AIMD; the margins of Table 8k are therefore superseded by Table 8l. The two limits stated here at the time (an agent trained only for windows up to 32 and for short transfers, and paths emulated only inside the client) are addressed in Sections 8.11 and 8.12.

Table 8j: Screening of window policies and caps (Selective Repeat; the neural cap-128 column is from the second screening, seeds 9501 to 9505, and is the same agent with no hand-over)

Screening, mean s (5 runs)	AIMD cap 32	AIMD tuned	CUBIC 128	Neural cap 32	Neural cap 64	Neural cap 128
Lossy 2%	7.98	8.05	17.45	4.36	4.26	4.04
Satellite-like	26.67	19.10	16.68	13.06	8.20	6.30
Fast	18.53	17.66	14.39	12.21	6.64	3.99

8.11 Corrected baselines, final configuration and validation on real sockets

Section 7.10 explains why the earlier comparison needs correcting: a retransmission timer that was too tight fired spuriously, and the AIMD baseline suffered from it more than the neural agent. With a floor of 1.3 times the smoothed RTT under the timer for every policy, tuned AIMD became much faster (satellite-like path 20.0 s to 11.4 s, fast path 17.3 s to 8.2 s), so the margins of Section 8.10 shrank. In the ablation of Section 8.12, removing the floor makes tuned AIMD 41% slower on random paths and the neural agent only 2% slower with twice the retransmissions. Two further additions were found with the virtual-time sender of Section 8.12 and are now part of the neural agent's default: classic slow start up to 64 packets before the policy takes over (it ends at the first loss or when the round-trip time rises by 30%), and pacing of new packets at twice the window divided by the smoothed RTT. The hand-over to AIMD at 10% loss per interval is kept as a safety net.

Table 8l is the final comparison: the C transport with Selective Repeat, a real clock and a real server process, the path emulated in the client, 10 paired runs per path (5 on the hostile path, seeds 9901 onward), 95% intervals. Tuned CUBIC (cap 128) is added as a second strong baseline. The neural agent is the fastest policy on the four ordinary paths. Paired against tuned AIMD it is significantly faster on all four (15%, 49%, 52% and 52% less time). Paired against tuned CUBIC it is significantly faster on the lossy path (74%) and the satellite-like path (34%), and not significantly different on the 5G-like and fast paths, where CUBIC is a strong competitor; CUBIC is however twice as slow as AIMD on the lossy path (13.9 s against 7.1 s), because it treats every random loss as congestion. On the hostile 10% loss path no policy is significantly different from another (the intervals are wide with five runs). Retransmissions are now low on the 5G-like, satellite-like and fast paths (3, 6 and 4 packets, against 33, 3 and 4 for tuned AIMD), and the 262 retransmissions of Table 8k no longer occur.

Table 8l: Final comparison, mean completion time (s), real clock (eval_final.sh; all differences against tuned AIMD are significant except the hostile path)

Path	AIMD cap 32	AIMD tuned	CUBIC tuned	Neural (final)	Neural vs tuned AIMD
5G-like, 1,200 packets	1.41±0.18	1.09±0.19	1.19±0.52	0.92±0.04	-15%
Lossy 2%, 1,200 packets	7.98±0.46	7.05±1.02	13.92±4.16	3.62±0.25	-49%
Satellite-like, 1,200 packets	17.93±1.69	11.36±4.38	8.26±2.67	5.46±0.38	-52%
Fast, 3,000 packets	14.55±0.88	8.20±2.29	6.52±3.16	3.92±0.17	-52%

Path	AIMD cap 32	AIMD tuned	CUBIC tuned	Neural (final)	Neural vs tuned AIMD
Hostile 10% loss, 300 packets (5 runs)	7.54±1.90	8.34±1.78	12.32±3.00	8.93±6.25	+7% (n.s.)

A weakness of everything so far was that the path was emulated inside the client. To remove that dependence a small impairment proxy was written in C (src/impair.c). It sits between client and server on real UDP sockets and applies random loss, delay with jitter, and a bottleneck with a drop-tail queue, with the client started with --emu 0 so that it adds nothing itself. Table 8m repeats the comparison through the proxy (8 paired runs per path, seeds 9501 onward). The agreement with Table 8l is close, and the conclusion is the same: the neural agent is the fastest policy on every path, significantly faster than tuned AIMD by 17%, 53%, 45% and 39% on the four ordinary paths, and not significantly different on the hostile path. This is still not a kernel network, because the proxy runs on loopback in user space; the netem_baselines.sh script remains the way to repeat the comparison with tc/netem and kernel TCP on a Linux host with root access, which was not available while this report was written.

Table 8m: Mean completion time (s) through the impairment proxy on real sockets (eval_proxy.sh)

Path (8 runs, real UDP sockets)	AIMD cap 32	AIMD tuned	Neural (final)	Neural vs tuned AIMD
5G-like	1.39±0.14	1.08±0.09	0.90±0.02	-17%
Lossy 2%	8.17±0.48	7.39±1.23	3.51±0.40	-53%
Satellite-like	16.92±1.66	9.50±2.30	5.26±0.25	-45%
Fast	14.21±0.99	6.20±1.62	3.76±0.18	-39%
Hostile 10% loss	8.74±1.91	8.28±1.93	8.10±4.59	-2% (n.s.)

8.12 A virtual-time version of the C sender: calibration, generalisation, ablation and training

The simulator of Sections 8.3 to 8.8 was written in Python and was never fully calibrated to the C code, a limitation recorded in Section 12. It has been replaced as the source of evidence by a virtual-time mode of the C sender itself (smart_client --vt 1). The sender code is unchanged; the server is replaced by an in-process receiver with the same Selective Repeat behaviour as server.c (256-packet window, 32-byte bitmap), and the clock advances by 0.35 ms per loop iteration, which is about what the real loop does with its 0.3 ms sleep. A 20-second transfer takes about 20 ms, results are reproducible from the seed, and there is no gap between simulator and implementation to calibrate. As a check, 40 virtual-time seeds per cell were compared with

the real-clock runs of the same policies on the five paths (web/vt_calib.py): in all 15 cells (5 paths by 3 policies) the two means agree within their 95% intervals.

Table 8n repeats the comparison with 100 paired seeds per path, which gives intervals about five times narrower than 10 real runs. The neural agent is significantly faster than tuned AIMD on the four ordinary paths (15%, 48%, 42% and 41%) and than tuned CUBIC (17%, 69%, 19% and 22%); on the hostile path it is 8% faster than tuned AIMD, which is not significant, and 45% faster than CUBIC, which is.

Table 8n: Mean completion time (s) in virtual time, 100 paired seeds per path (web/vt_named.py)

Path	AIMD cap 32	AIMD tuned	CUBIC tuned	Neural	vs AIMD	vs CUBIC
5G-like	1.32±0.04	1.09±0.04	1.11±0.09	0.93±0.01	-15%	-17%
Lossy 2%	7.93±0.18	7.18±0.24	12.08±1.05	3.76±0.14	-48%	-69%
Satellite-like	18.22±0.57	9.24±0.83	6.58±0.55	5.33±0.14	-42%	-19%
Fast	13.90±0.20	6.41±0.45	4.80±0.40	3.75±0.04	-41%	-22%
Hostile 10%	8.88±0.68	8.75±0.79	14.81±1.87	8.08±0.87	-8% (n.s.)	-45%

Generalisation. Five hand-picked paths can flatter any policy that was tuned while looking at them, so the comparison was repeated on 100 paths drawn at random from wide ranges: one-way delay 5 to 150 ms, jitter up to a quarter of the delay, random loss 0.05% to 8% (log-uniform), bottleneck 300 to 4,000 packets per second (log-uniform), queue 20 to 300 packets, and 600, 1,200 or 2,000 packets. The tuning choices below were made on a different set of random paths (seed 1001); the test set (seed 424242) was used only for the numbers in Table 8o. Two seeds were run per path. On the test set the neural agent needs 0.534 times the time of tuned AIMD (geometric mean; 47% faster), is faster by more than 5% on 89 of 100 paths and slower by more than 20% on none, with a worst case of 1.13. Tuned CUBIC is 0.984 on average but very uneven: slower by more than 20% on 27 paths, and 2.3 times slower in the worst case.

Table 8o: Completion time relative to tuned AIMD on 100 random held-out paths (web/vt_random.py)

Policy (100 random paths)	Geometric-mean time ratio	Median	Worst	Faster on	More than 20% slower on	Retransmissions per packet
AIMD tuned (reference)	1.000	1.000	1.00	-	-	0.024
CUBIC tuned	0.984	0.949	2.32	51/100	27/100	0.035
Neural (final)	0.534	0.508	1.13	89/100	0/100	0.028

Policy (100 random paths)	Geometric-mean time ratio	Median	Worst	Faster on	More than 20% slower on	Retransmissions per packet
Neural, evolution-strategy weights	0.521	0.482	1.57	88/100	1/100	0.024

Ablation in the real sender. Table 8p switches off one ingredient at a time on the same 100 test paths (different seeds, so the final configuration reads 0.510 here and 0.534 above; the spread between seed sets is about that size). The window cap matters most: with a cap of 32 the agent is only 16% faster than tuned AIMD and is more than 20% slower on 21 paths, which is the original configuration's weakness (the original settings together give 0.859). Pacing is next (7.6% in time and 39% fewer retransmissions), then slow start before the policy (3%). The hand-over to AIMD does not change the average on these paths (0.506 without it), as expected, since it exists for the hostile case; its value there is that it prevented the Go-Back-N collapse of Section 8.8. The timer floor is the largest single factor for AIMD (41%) and halves the agent's retransmissions.

Table 8p: Ablation in the C sender (web/vt_ablate.py, results/vt_ablation.txt)

Variant (100 random test paths)	Time ratio to tuned AIMD	Versus final	More than 20% slower	Retx per packet
Tuned AIMD (reference)	1.000	1.96	0	0.020
Tuned AIMD without the timer floor	1.412	2.77	49	0.035
Neural agent, final configuration	0.510	1.00	1	0.028
without slow start before the policy	0.527	1.03	4	0.028
without pacing	0.549	1.08	4	0.046
without the AIMD hand-over	0.506	0.99	1	0.028
window cap 32 instead of 128	0.837	1.64	21	0.016
without the timer floor	0.519	1.02	2	0.053
original settings (RUDP_CLASSIC=1)	0.859	1.68	25	0.020

Tuning sweeps. On the tuning paths (80 paths, 2 seeds each, time relative to tuned AIMD) the agent without slow start scores 0.596 and is more than 20% slower on 11 of 160 runs. Slow start to 32, 48, 64, 96 and 128 packets gives 0.594, 0.573, 0.557, 0.558 and 0.563, and 64 was chosen. With slow start to 64 packets, hand-over thresholds of 10% (the default), 15%, 20% and none give 0.557, 0.547, 0.549 and 0.551, so the threshold matters little on ordinary paths; lower thresholds hurt (with slow start to 128, 5% gives 0.582 against 0.563 at 10%, and without slow

start 3% gives 0.649 against 0.596). Pacing at 2, 1.25 and 1 times cwnd/srtt gives 0.529, 0.544 and 0.558 against 0.547 unpaced, with retransmissions of 0.033, 0.032, 0.027 and 0.043 per packet; pacing is neutral for AIMD (0.999 at a factor of 2) so it does not favour the agent.

Training in the real sender. Because the virtual-time sender is the C code, it can be used as the training environment. An evolution strategy (antithetic sampling, rank-shaped fitness, Adam; web/es_train.py) tuned the agent against fitness = mean of log(time relative to tuned AIMD) plus 1.5 times the retransmissions per packet, on fresh batches of random training paths. Updating only the last layer (165 weights; 24 antithetic pairs, 30 paths per generation, 100 generations, about 11 minutes) improved the validation score from -0.506 to -0.564. On the untouched test set the gain was 2.4% in time (0.534 to 0.521) with one path 57% slower than AIMD, so it was not adopted. Updating all 1,413 weights for 120 generations gave no gain beyond noise. The conclusion is that the original weights, trained in the Python simulator with a window cap of 32, already generalise to a cap of 128 and to the C environment; this answers the earlier question whether the agent needed retraining for larger windows (it did not, for these paths).

8.13 Fairness on real sockets

Section 8.6 measured fairness only in the Python simulator. It was repeated with the C transport on real sockets: two sender processes, each with its own server process, send 2,500 packets each through the multi-flow mode of the impairment proxy (src/impair.c), which gives both flows one shared bottleneck (30 ms one-way delay, 1,000 packets per second, queue 60, 0.1% random loss; 6 runs per pairing, eval_fair.sh). The share of a flow is the number of its packets that passed the bottleneck by the time the first flow finished, and Jain's index is (a+b)^2 / (2(a^2+b^2)), where 1.0 is an equal split and 0.5 is the worst case for two flows.

Table 8q: Share of a shared bottleneck between two flows on real sockets (eval_fair.sh)

Flow A against flow B	Share of A	Share of B	Jain index	Runs
AIMD tuned against AIMD tuned	49%	51%	0.987	6
Neural against neural	50%	50%	0.968	6
Neural against AIMD tuned	68%	32%	0.876	6
Neural against AIMD cap 32 (original)	76%	24%	0.781	6
CUBIC tuned against AIMD tuned	61%	39%	0.933	6

Two neural flows split the link evenly, as do two AIMD flows. Against AIMD the neural agent takes about two thirds of the link (68%) and against the original cap-32 AIMD about three quarters (76%); tuned CUBIC takes 61% against tuned AIMD. The neural agent is therefore a

more aggressive neighbour than CUBIC, though no flow was starved: the other flow always finished. This agrees in direction with the simulator study of Section 8.6. Part of the 76% is simply that the cap-32 flow cannot use more than 32 packets in flight, but the 68% against an equally tuned AIMD flow is a real cost of the agent's speed. The agent was not trained to be fair, and coexistence with TCP was not tested; both would be needed before it could be deployed on a shared link.

8.14 Engineering hardening and tools

Alongside the performance work the implementation was hardened and given the tooling expected of a maintained project. The checksum is now CRC-16 and the parser is fuzzed under sanitizers (Section 7.8); the handshake no longer hangs when its last ACK is lost (Section 7.9); and the terminal chat and the web gateway now use the same 256-packet window and 32-byte ACK bitmap as the C transport, which was verified by an automatic interoperability test between the Python gateway and the C chat at 20% loss in both directions (Section 7.11). A GitHub Actions workflow builds with warnings as errors, runs the whole test suite and the fuzzer on every push. The architecture is described in docs/ARCHITECTURE.md and the project carries an MIT licence.

File transfer. The sender can now send a real file (smart_client --file), and the server writes the delivered byte stream to a file (RUDP_OUT) and reports its length and an FNV-1a hash, so a transfer can be checked end to end. filetransfer_demo.sh sends a random file through the impairment proxy with three policies and compares the hashes. In every run the received file was byte-identical. Table 8r shows the goodput on two paths: the neural agent halves the time of tuned AIMD on the lossy path (2 MB) and needs 42% of its time on the fast path (4 MB). On the lossy path it also sends more retransmissions (172 against 77). The test suite also sends 400 kB through 5% loss and compares the files.

Table 8r: File transfer through the impairment proxy (filetransfer_demo.sh; byte-identical means equal length and FNV-1a hash)

File and path	Policy	Time	Goodput	Retransmissions	Received file
2 MB, lossy 2%	AIMD cap 32	26.6 s	0.08 MB/s	82	identical
	AIMD tuned	24.2 s	0.08 MB/s	77	identical
	Neural	11.9 s	0.17 MB/s	172	identical
4 MB, fast	AIMD cap 32	34.6 s	0.12 MB/s	6	identical
	AIMD tuned	21.0 s	0.19 MB/s	7	identical

File and path	Policy	Time	Goodput	Retransmissions	Received file
	Neural	8.9 s	0.45 MB/s	6	identical

Raw speed. With the emulated path switched off (bench_loopback.sh, 4,096 packets of 40 bytes on the loopback interface, median of 9 runs) the C sender sustains about 40,000 packets per second with AIMD and 42,000 with the neural agent, which is now as fast as AIMD because of the slow-start phase; tuned CUBIC reaches 17,000 because its window grows slowly when the round trip is 0.4 ms. These are small payloads on one machine and say only that the sender is not the bottleneck at the rates used in this report.

8.15 A fairer neighbour, bursty cross traffic, and a multi-client server

Three of the limitations listed in the previous version of this report were worked on. **Fairness.** The agent has an optional polite mode (smart_client --polite X): when the round-trip ratio is at least X and the queueing delay is larger than twice the measured RTT deviation (so that jitter is not mistaken for a queue) and the window is above 12 packets, the agent backs off by a factor of 0.85 instead of following the network. Two flows, the neural agent against tuned AIMD, were run through the shared-bottleneck proxy, 8 runs per setting.

Setting	Agent share	Jain index	Agent time	AIMD time	Runs
Default (no polite mode)	70%	0.860	4.7 s	7.8 s	8
--polite 1.2	49%	0.945	5.3 s	5.4 s	8
--polite 1.3 (recommended)	52%	0.936	5.1 s	5.8 s	8
--polite 1.4	58%	0.889	4.8 s	6.7 s	8
--polite 1.5	59%	0.896	5.1 s	7.1 s	8

Polite mode with X = 1.3 brings the share from 70% to about half and the Jain index from 0.86 to 0.94, and it also shortens the competing AIMD flow from 7.8 s to 5.8 s. The price on a single flow is speed: over the 100 random held-out paths the geometric-mean time ratio to tuned AIMD rises from 0.534 to 0.576, the agent is still faster on 89 paths, but 3 paths are now more than 20% slower and the worst is 2.95 times slower (the default has none and 1.13). A first version without the jitter test and the window floor stalled on a high-jitter path (76 s against 25 s for AIMD), which is why those two conditions exist. The mode is therefore off by default. Individual runs vary (the agent's share ranged from 21% to 58% in a four-run check), so the share is an average, not a guarantee.

Bursty cross traffic. The proxy gained a cross-traffic source (--cross ON:OFF:PPS) that injects exponentially distributed bursts into the same bottleneck (here 300 ms on, 500 ms off, 700 packets per second on a 1,000 packet per second link). One flow of 2,500 packets, 8 paired runs per policy, real sockets:

Policy	Time (s)	95% interval	Ratio to tuned AIMD	Retransmissions
AIMD, cap 32	6.73	+- 0.38	1.18	3
AIMD, tuned	5.73	+- 0.51	1.00	70
CUBIC, tuned	4.55	+- 0.60	0.79	98
Neural agent	4.87	+- 0.37	0.85	200
Neural agent, polite 1.3	4.92	+- 0.54	0.86	151

This is a less favourable result than the clean paths. The agent is about 15% faster than tuned AIMD, but it is not distinguishable from CUBIC, whose interval overlaps, and it retransmits about three times as often as tuned AIMD (200 against 70 packets of 2,500) because bursts overflow the queue while the window is large. Three attempts to reduce this (a window cut of 0.7 or 0.5 on a timeout, and handing over to AIMD at 3% loss instead of 10%) all gave 4.8 to 4.9 s, so none was adopted. The claim that the agent is much better than tuned AIMD should therefore be limited to the paths without competing bursts; with them, a simple CUBIC is as good.

An attempt to learn fairness (negative result). The virtual-time sender gained a simulated, always-busy AIMD competitor on the same bottleneck (--vcomp), and the evolution-strategy trainer was given a penalty on taking more than half of the link. Two runs (penalty weight 3 for 60 generations, 12 for 150) were tested on 40 held-out random paths: the agent's mean share stayed at 0.59 to 0.60 (AIMD against the same competitor takes 0.47), and the time ratio to AIMD stayed between 0.52 and 0.56. The rule-based polite mode, which works on real sockets, also barely changes the share in this simulator (0.59 to 0.58), so the simulated competitor does not reproduce the real two-flow behaviour well enough to train against. The original weights were kept (results/es_fair/SUMMARY.txt).

Training with cross traffic. The virtual-time sender also gained the same bursty cross traffic (--vcross) and the trainer a mode that puts it on half the training paths. After 150 generations the new weights (results/dqn_cross.txt) reduced the geometric-mean time ratio to tuned AIMD from 0.530 to 0.502 on 60 held-out paths with cross traffic (0.635 to 0.619 on a second set of 60), and on the five named paths they were 12% faster on the lossy path, 3% and 1% faster on two others, and 3% slower on the 5G-like path (retransmissions 4 to 21). On real sockets with cross traffic the mean fell from 4.87 s to 4.53 s over 8 paired runs, but the difference (341 ms,

interval of 706 ms) is not significant and retransmissions rose from 200 to 218. It is therefore offered as an option, not made the default, and the cross-traffic limitation above stands.

Multi-client server. The server keeps one session per client address (up to 16 at once, the least recently active is recycled), so several senders can use one server process; a retransmitted FIN after the connection closed is now acknowledged again. A regression test sends two files from two senders at the same time and checks that both arrive byte-identical (tests/test_multiclient.sh, part of make test).

8.16 The sender as a library

The sender had grown inside one command-line program. It was refactored into a C library, librudp (src/rudp_sender.c, public header src/rudp.h), with a small API: rudp_options_default, rudp_send (a buffer), rudp_send_file, rudp_send_test_pattern and rudp_strerror, plus option and statistics structures. The command-line program (smart_client) is now a thin front end that fills the options and prints the RESULT line, so every evaluation script works unchanged. The library can be built with make lib, installed with make install, and used as in examples/send_buffer.c.

Because every number in this report comes from that code, the refactor was checked by equivalence: 34 deterministic virtual-time results (the AIMD, CUBIC, neural, polite, table and Go-Back-N policies on two paths with two seeds, plus runs with a competitor and with cross traffic) were recorded before the change and compared after it; all 34 lines are byte-for-byte identical. A new test (tests/test_api.sh) sends two payloads one after the other from one process through the API, with 5% emulated loss, and compares the sender's FNV-1a hash with the server's, which also shows that no state leaks between calls (a stale selective-acknowledgment array would have corrupted the second transfer). Limits: one transfer per call (connect, send, close), at most 32 MB, IPv4 only, and global state, so it is not thread-safe.

8.17 Validation on a kernel network (tc/netem)

Every earlier real-socket result used an emulated path inside the program or a user-space proxy. The comparison was therefore repeated on the Linux kernel itself: tc/netem on the loopback interface of a WSL2 host (run by the first author). Each protocol sent 500 packets of 512 bytes (RUDP sends a real 256,000-byte file, so the bytes are the same as for kernel TCP and plain UDP) over four paths, five runs each (three on the satellite path), one after the other (netem_full.sh; corrected results in results/netem_corrected.txt, earlier runs in results/netem_summary.txt). Delay, jitter, random loss and reordering are applied by a netem stage, and the bottleneck is a separate token-bucket stage with a drop-tail queue of 50 to 100 packets, so that the queue limit does not count packets in flight.

The first runs were wrong, and a fourth mistake was found later. They used the netem option rate with limit 60. In netem the limit also counts packets held in the delay line, so on the

satellite path (about 200 packets in flight) any window above 60 packets overflowed the qdisc. That produced 13 to 20 percent retransmissions for AIMD and the neural agent, made the neural agent tie with AIMD on the satellite path, and I first blamed reordering for it. The same runs also compared against untuned AIMD (initial window 1, threshold 16) rather than the tuned AIMD used elsewhere, and TCP was sent 1,024-byte payloads against 512 for RUDP. A fourth mistake was found only when the coexistence run of Section 8.22 reported a goodput above the link capacity: without --file the sender uses synthetic 40-byte packets, so RUDP had been carrying about twelve times fewer bytes than TCP and UDP in every kernel run, including the runs described in the first version of this section as using the same bytes. All four problems were fixed, the whole kernel comparison was repeated with real 512-byte payloads, and the earlier runs are superseded; they are kept in results/netem_summary.txt with the correction. The virtual-time queue, which delivered packets strictly in the order sent, was also fixed (it now delivers by arrival time, so --reorder 1 works); results with the default in-order path are unchanged (make repro: geometric-mean ratio 0.534, faster on 89 of 100 paths).

Table 8s: Kernel netem with real 512-byte payloads, mean completion time of 500 packets, 5 runs per cell unless stated (TCP: kernel stack; AIMD tuned: window cap 128, threshold 128)

Path	TCP (s)	AIMD, tuned (s)	AIMD + initial window 10 (s)	Neural (s)	Neural / tuned AIMD	Welch p
Lossy (25 ms, 2% loss)	0.88	4.75	4.21	1.35	0.29	0.0002
5G-like (10 ms)	0.28	1.23	1.36	0.43	0.35	<0.0001
Satellite (150 ms), 3 runs	1.91	16.72	15.36	2.92	0.17	0.007
Reordering (50 ms, 25%)	0.94	4.06	4.24	1.64	0.40	0.09

Against tuned AIMD the neural agent is faster by 71%, 65% and 83% on the lossy, 5G-like and satellite paths, all significant, and by 60% on the reordering path, where the difference is not significant ($p = 0.09$) because tuned AIMD is bimodal there (1.1 to 6.4 s). A TCP-like initial window of 10 does not help AIMD on these paths. **Kernel TCP is faster than the neural agent on every path**: 1.5 times on the lossy, 5G-like and satellite paths and 1.7 times on the reordering path. Plain UDP delivered 26 to 36 percent of the packets. The repeat with real payloads gave nearly the same ratios as the flawed runs (for example 83% on the satellite path in both), which suggests that on these latency- and loss-bound paths the controller, not the byte count, decides the time; this was observed, not separately tested. The honest summary is that

the learned controller is clearly better than the AIMD baselines on a real kernel path, and that it does not match a mature kernel TCP, which has decades of tuning, a larger initial window handled in the kernel, and no user-space scheduling cost. Run-to-run variation on a shared laptop is large (the neural agent took 0.8 to 2.7 s on the lossy path), so differences of tens of percent between single cells should not be over-read.

8.18 Paths that change during a transfer

Every earlier test used a path whose bandwidth and delay stay constant. Experimental comparisons of learned and human-derived congestion control [14] motivate testing how a learned controller copes when the path changes during a flow. A new option, --dyn, was added to the virtual-time sender: it lists times at which the bottleneck rate and the one-way delay take new values. Four cases were run on a path of 1,500 packets/s, 30 ms one-way delay, 0.2% loss and a 60-packet queue (4,000 packets, 20 paired seeds, tools/dyn_eval.py): the rate drops to 300 packets/s after 0.8 s; the rate rises from 300 to 1,500 packets/s after 0.8 s; the one-way delay jumps from 30 to 120 ms after 0.8 s; and a satellite-like hand-over pattern in which every second the rate falls to 200 packets/s for 150 ms and the delay alternates between 25 and 45 ms.

Table 8t: Mean completion time (s) of 4,000 packets on paths that change during the transfer, ±95% interval, 20 paired seeds, virtual time

Case	AIMD, tuned	CUBIC	BBR-style	Neural
Bandwidth drop	13.58 ± 0.25	17.74 ± 1.37	13.39 ± 0.27	12.72 ± 0.18
Bandwidth rise	9.21 ± 0.88	15.10 ± 1.51	5.00 ± 0.14	4.32 ± 0.15
Delay jump	33.32 ± 2.67	27.54 ± 3.11	25.94 ± 2.29	19.80 ± 0.62
Hand-over pattern	12.84 ± 0.60	21.40 ± 1.51	10.79 ± 0.45	8.18 ± 0.42

The neural agent is fastest in all four cases, by 6% to 53% against tuned AIMD, although it was trained only on constant paths. The price is retransmissions: the neural agent resends 50 to 241 of 4,000 packets (1% to 6%) against 18 to 78 for tuned AIMD, because it probes upward quickly and overshoots the queue after a drop. The BBR-style policy (Section 8.19) is second on three of the four cases. The result supports the claim that the agent does not need a fixed path, but it is a simulation of four hand-made changes, not recorded traces; the hand-over pattern is a crude stand-in for a low-Earth-orbit link.

8.19 A model-based baseline (BBR-style)

The literature review had listed a model-based baseline as missing. A simplified version of BBR [17] was added to the sender (--mode bbr): it estimates the bottleneck bandwidth as the maximum delivery rate over the last ten round-trip times and the propagation delay as the

minimum RTT, starts with a gain of 2.885 until the bandwidth stops growing, drains the queue, then cycles the pacing gain through 1.25, 0.75 and six rounds of 1; the window is twice the estimated bandwidth-delay product and loss does not change it. It is not the Linux implementation: ProbeRTT is replaced by letting the minimum-RTT estimate expire after ten seconds, there is no ECN signal and no BBRv3 loss response, and the window cap of 128 applies. A first version collapsed in 2 of 20 runs at 10% loss, because one low bandwidth sample after a gap in ACKs poisoned the estimate (the transfer made no progress for 600 s); samples taken across a long gap are now discarded. A first version also took 163 s after a delay jump, because the stale minimum RTT kept the window far too small until the expiry was added. Both defects were found by the tests described here and are fixed.

Table 8u: Mean completion time (s) of 3,000 packets on the five named paths, ±95% interval, 20 paired seeds (test seed 424242 onward), virtual time

Path	AIMD, tuned	CUBIC	BBR-style	Neural
5G-like	2.78 ± 0.17	7.14 ± 0.96	2.26 ± 0.02	2.21 ± 0.03
Lossy 2%	19.79 ± 0.82	53.25 ± 1.70	8.18 ± 0.69	9.76 ± 0.58
Satellite-like	32.76 ± 3.03	26.62 ± 3.21	16.78 ± 0.44	11.18 ± 0.45
Fast	7.83 ± 1.42	12.45 ± 1.77	6.23 ± 0.06	3.72 ± 0.07
Hostile 10% loss	94.73 ± 7.57	208.81 ± 11.09	92.29 ± 5.47	99.38 ± 10.78

Table 8v: Mean completion time (s) of 3,000 packets on a 1,000 packets/s, 40 ms path as random loss rises, 20 paired seeds, virtual time (full sweep in results/bbr_compare.txt)

Random loss	AIMD, tuned	CUBIC	BBR-style	Neural
0.1%	5.46 ± 0.89	11.16 ± 1.35	4.36 ± 0.05	3.63 ± 0.09
1%	18.61 ± 1.00	40.12 ± 2.61	5.70 ± 0.33	5.38 ± 0.26
2%	28.41 ± 1.19	66.03 ± 3.26	9.52 ± 1.40	8.34 ± 0.44
5%	47.61 ± 1.15	101.85 ± 3.12	35.40 ± 1.79	27.89 ± 2.50
10%	93.72 ± 5.03	209.86 ± 11.09	100.13 ± 6.30	93.81 ± 7.71

The BBR-style policy is a strong baseline: it beats tuned AIMD on four of the five named paths and ties on the hostile one, and it is the fastest policy on the lossy 2% path (8.18 s against 9.76 s for the neural agent, a difference larger than the intervals). The neural agent is faster on the satellite-like and fast paths (11.18 against 16.78 s and 3.72 against 6.23 s) and level on the 5G-like path. In the loss sweep the neural agent is ahead of or level with the BBR-style policy at

every loss rate (the gap is clear at 0.1%, 0.5%, 3% and 5%, within the intervals at 1%, 2% and 10%). BBRv3 is reported to collapse above about 2% random loss (a claim from the literature search for this section whose source the authors still have to verify); the simplified policy used here does not, so that finding does not carry over to this implementation, and no claim is made about the Linux BBR. The earlier claim that the neural agent is better than the established alternatives therefore has to be read against this baseline: it holds on four of five named paths in this simulator, not on the 2% lossy path.

8.20 Fairness between neural flows with unequal round-trip times

Section 8.13 measured neural against neural with equal round-trip times (Jain index 0.97). Two flows rarely have equal round-trip times, and whether the fairness of learned controllers generalizes to other conditions is the subject of recent work [15]. The impairment proxy was given an --xdelay option that adds delay to the ACK path of the second flow only, and two flows sharing a 1,000 packets/s, 30 ms bottleneck were run, six pairs per cell (eval_fair_rtt.sh, real sockets). Flow B has the longer round-trip time.

Table 8w: Bandwidth shares of two identical flows with unequal round-trip times (packets through the bottleneck when the first flow finished); six pairs per cell, real sockets, user-space proxy

Pair	Extra RTT of B	Share of A (short RTT)	Share of B	Jain index	Pairs
AIMD / AIMD	0 ms	45%	55%	0.964	6
Neural / neural	0 ms	48%	52%	0.964	6
AIMD / AIMD	60 ms	74%	26%	0.804	6
Neural / neural	60 ms	67%	33%	0.885	6
AIMD / AIMD	150 ms	88%	12%	0.641	6
Neural / neural	150 ms	71%	29%	0.840	6

Both policies favour the short-round-trip flow, as window-based control does, but the neural pair is closer to even: the Jain index at +150 ms is 0.84 against 0.64 for AIMD, and the long-RTT flow keeps 29% of the link against 12%. With six pairs per cell the differences at +60 ms are within the noise and only the +150 ms difference is clear. The agent was not trained for fairness; the effect is a side-effect of its rate-of-growth actions being less tied to the round-trip time than additive increase.

8.21 What the neural network does, and how far it can be trusted

Interpretability and robustness were examined with the logged decisions of the agent (about 5,500 decisions on the five named paths for training and 5,500 on disjoint seeds for testing; tools/interpret.py, tools/interp_eval.py, results/interp/). Three findings stand out.

- **It is a bang-bang policy that depends on every input.** On the visited states it multiplies the window by 1.5 in 63% of decisions and halves it in 17%; it chose the 1.15 factor in under 0.5% of decisions. Shuffling any single input across states changes the decision in 41% (RTT growth), 21% (loss), 41% (window), 45% (throughput share) and 16% (last action) of states, so no input can be dropped. A change of +0.1 in the loss input alone flips 37% of decisions.
- **A small rule set reproduces its behaviour, but not its decisions.** A decision tree fitted to the network agrees with it on 77% of unseen states at depth 4 (16 leaves), 79% at depth 5 (32 leaves) and 87% at depth 8 (165 leaves). The tree is nevertheless as fast as the network: Table 8x shows that the depth-5 tree finishes within the intervals of the network on four of five paths, and is 9% slower on the 5G-like path (2.41 against 2.21 s). The network therefore contains a lot of decision detail that does not matter for the transfer time; it does not show that the tree is a faithful explanation.
- **It is not formally verified.** Interval bound propagation through the 5-32-32-5 network can certify that the chosen action cannot change for any input within ±0.001 of the observed state for 43% of states, within ±0.003 for 8%, within ±0.01 for 0.2% and within ±0.03 for none. Random sampling of 200 points inside the same boxes found the action stable in 99%, 96%, 83% and 61% of boxes, so the true robustness is far better than the certificate shows, but it is not proven. Learning a controller together with formal certificates has been proposed [16]; this one has none.

Table 8x: Mean completion time (s), 20 paired seeds, virtual time. Tree = distilled decision tree of the given depth; Noise = Gaussian noise of the given standard deviation added to the four measured inputs

Path	Network	Tree d8	Tree d5	Noise 0.02	Noise 0.05	Noise 0.1	Noise 0.2
5G-like	2.21	2.26	2.41	2.25	2.25	2.32	2.47
Lossy 2%	9.76	10.19	9.24	10.76	10.77	10.28	9.93
Satellite-like	11.18	11.31	11.63	11.23	11.36	11.36	11.63
Fast	3.72	3.72	3.72	3.77	3.80	3.83	3.79
Hostile 10% loss	99.4	93.4	104.6	93.0	97.3	88.8	95.6

Noise on the measured inputs (RTT growth, loss, throughput share, window; the inputs range roughly from 0 to 1) up to a standard deviation of 0.2 changes the completion time by at most 12% (the 5G-like path, 2.21 to 2.47 s) and is within the intervals elsewhere. The agent is therefore tolerant of measurement noise in practice, although the formal certificate is weak.

8.22 Coexistence with kernel TCP

The remaining open question in Section 8.15 was whether the neural agent starves a kernel TCP flow that shares its bottleneck. A bulk kernel TCP flow (CUBIC, the WSL default) and one RUDP transfer of 4,000 real 512-byte packets ran at the same time through the netem and token-bucket path of Section 8.17 (netem_coexist.sh, web/tcp_bulk.py, run by the first author, three runs per cell). The goodput of each flow was measured over the window in which both were running; the control replaces RUDP by a second kernel TCP flow. A first set of runs was discarded because RUDP again sent synthetic 40-byte packets (Section 8.17).

Table 8y: Share of the summed goodput taken by the second flow when a kernel TCP flow and an RUDP flow share the bottleneck, three runs per cell (range shown)

Path	TCP vs TCP: second flow	AIMD vs TCP: AIMD share	Neural vs TCP: neural share	RUDP alone, AIMD (Mbit/s)	RUDP alone, neural (Mbit/s)
5G-like (12 Mbit/s)	48% to 54%	8%	18% to 30%	1.60	7.26
Lossy (6 Mbit/s)	36% to 59%	15% to 17%	27% to 32%	0.48	1.03
Satellite (3 Mbit/s)	34% to 46%	5%	27% to 41%	0.12	1.25

The neural agent does not starve kernel TCP. It takes less than half of the link on every path (mean shares 24%, 30% and 34%), and TCP keeps 7.4 to 8.8 Mbit/s of the 12 Mbit/s link on the 5G-like path, 2.2 to 2.8 Mbit/s of 6 on the lossy path and 1.4 to 1.9 Mbit/s of 3 on the satellite path. Tuned AIMD is squeezed out much more (5% to 17%), which matches its very low speed when alone. On this evidence the agent is a polite neighbour of kernel TCP, but only because it is slower than TCP: its single-flow goodput is below TCP's on the 5G-like and lossy paths. The earlier worry (Section 8.15) was about sharing with AIMD flows, where it takes about two thirds. Limits: three runs per cell on one laptop, so shares are good to roughly ten points (the TCP-against-TCP control alone ranges from 36% to 59% on the lossy path); the TCP-alone figure on the satellite path (1.13 Mbit/s) is lower than the TCP goodput measured next to RUDP because it includes slow start; one TCP congestion control (CUBIC) and no multi-machine path.

9. NETWORK DESIGN AND CISCO PACKET TRACER

To place the transport protocol inside a realistic network, a four-router topology was built in Cisco Packet Tracer (Figures 23 and 24). A campus LAN with two VLANs reaches a server LAN across a routed core of three routers (R1, R2 and R3) connected in a triangle, and a branch LAN hangs off a fourth router (R4) behind R2. All routers run OSPF [13]. The RUDP client would run on a campus PC and the RUDP server on PC2, so every RUDP segment crosses the routed core as an ordinary UDP datagram to port 8888. In Packet Tracer the routers carry the device labels Router1, Router2, Router3 and Router0 for R1, R2, R3 and R4; the host names configured on them are R1 to R4.

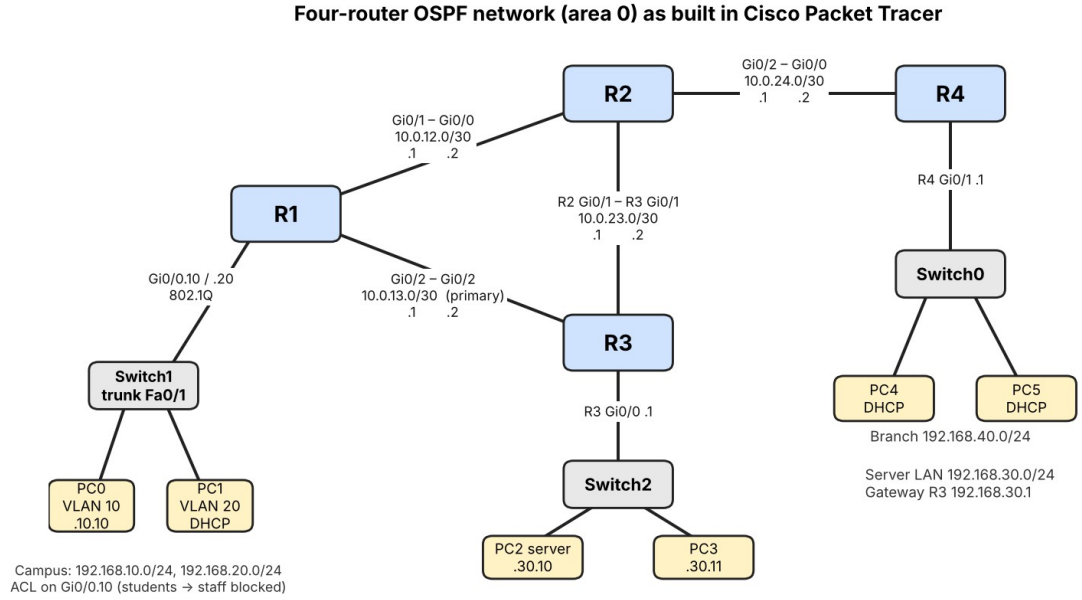


Figure 23: *Four-router OSPF network as built: two campus VLANs behind R1, a server LAN behind R3, a branch LAN behind R4, with the interface used on each link*

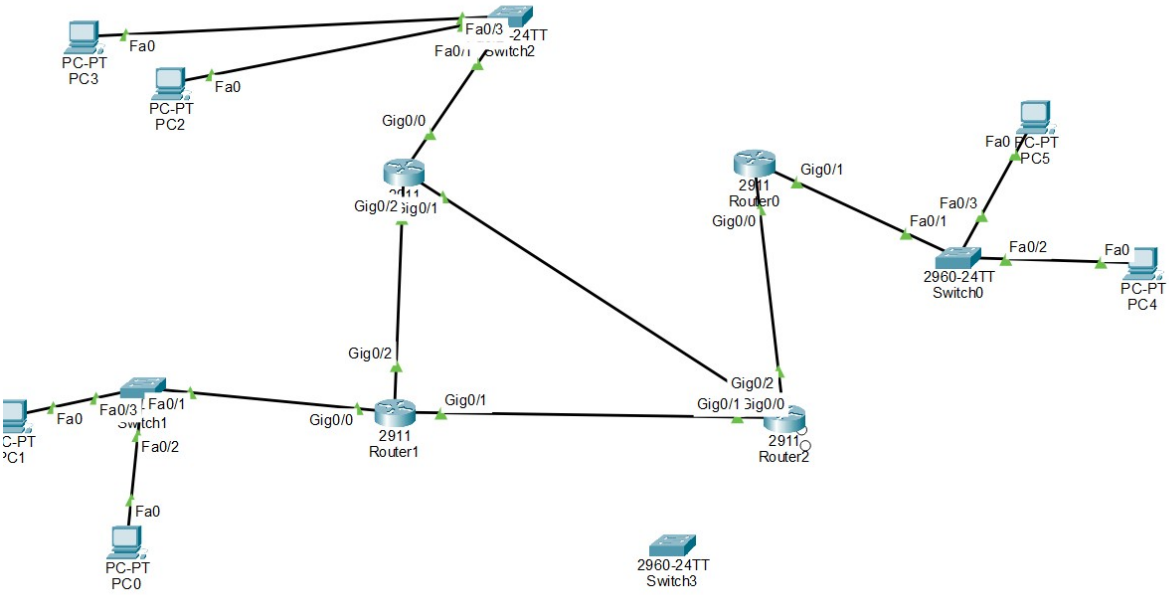


Figure 24: *The same network in Cisco Packet Tracer with port labels shown*

9.1 Addressing and configuration

Table 9: Addressing plan

Link	Subnet	Notes
R1 Gi0/1 - R2 Gi0/0	10.0.12.0/30	R1 .1, R2 .2
R2 Gi0/1 - R3 Gi0/1	10.0.23.0/30	R2 .1, R3 .2
R1 Gi0/2 - R3 Gi0/2	10.0.13.0/30	R1 .1, R3 .2 (primary path)
R2 Gi0/2 - R4 Gi0/0	10.0.24.0/30	R2 .1, R4 .2
VLAN 10 (students)	192.168.10.0/24	Gateway R1 .1; DHCP pool VLAN10; PC0 192.168.10.10
VLAN 20 (staff)	192.168.20.0/24	Gateway R1 .1; DHCP pool VLAN20; PC1 192.168.20.11
Server LAN	192.168.30.0/24	Gateway R3 .1; PC2 192.168.30.10, PC3 192.168.30.11
Branch LAN	192.168.40.0/24	Gateway R4 .1; DHCP pool BRANCH; PC4 192.168.40.11

Switch1 carries VLANs 10 and 20 to R1 over a trunk, and R1 routes between them with router-on-a-stick sub-interfaces (802.1Q encapsulation). R1 and R4 act as DHCP servers for the

campus VLANs and the branch LAN. All four routers run OSPF process 1 in area 0 with router IDs 1.1.1.1 to 4.4.4.4. An extended access list named BLOCK-STUDENT-TO-STAFF on R1 sub-interface Gi0/0.10 denies traffic from the student VLAN to the staff VLAN and permits everything else. Because the replies of connections started from the staff VLAN toward students also have to pass back through that interface, the policy is effectively two-way between those VLANs, while both VLANs can still reach the server. During the build, the cabling of R2 was found by CDP (show cdp neighbors) to differ from the first plan, and the R2 addresses were changed to match the real cables.

9.2 Test plan and observed results

Table 10 lists the checks performed on the built network. The expected values come from the OSPF cost rules (a Gigabit Ethernet link has cost 1), and the right-hand column gives what was observed in Packet Tracer. R1 reaches the server LAN with metric 2 through R3 directly and the branch LAN with metric 3 through R2 (Figure 25). The first ping of every run was lost while ARP resolved, which is normal.

Table 10: Packet Tracer tests and observed results

No.	Check	Expected	Observed
1	R2: show ip ospf neighbor	Three FULL neighbours	1.1.1.1, 3.3.3.3 and 4.4.4.4 reached FULL
2	R1: show ip route ospf	192.168.30.0 metric 2, 192.168.40.0 metric 3	As expected: via 10.0.13.2 and via 10.0.12.2
3	PC0: tracert 192.168.30.10	192.168.10.1, 10.0.13.2, server	Exactly these three hops
4	PC4 (DHCP): ping the server	Reachable through R4, R2, R3	4 of 4 replies, TTL 125
5	PC0: ping a VLAN 20 host and the server after the access list	Staff host blocked, server reachable	Staff host: destination unreachable from 192.168.10.1; server still reachable (4 of 4, TTL 126)
6	Shut R1 Gi0/2 during ping -t to the server	A few losses, then recovery via R2	70 of 71 replies; tracert via 10.0.12.2 and 10.0.23.2
7	Restore R1 Gi0/2	Neighbour returns	R1-R3 returned to FULL

```
Neighbor ID     Pri   State           Dead Time   Address         Interface
3.3.3.3           1   FULL/DR         00:00:36    10.0.13.2       GigabitEthernet0/2
R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     10.0.12.1       YES manual up                    up
GigabitEthernet0/2     10.0.13.1       YES manual up                    up
Vlan1                  unassigned      YES unset  administratively down down
R1#
01:06:43: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from LOADING to FULL,
Loading Done

01:09:03: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from FULL to DOWN,
Neighbor Down: Dead timer expired

01:09:03: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from FULL to DOWN,
Neighbor Down: Interface down or detached

01:09:03: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/2 from FULL to DOWN,
Neighbor Down: Dead timer expired

01:09:03: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/2 from FULL to DOWN,
Neighbor Down: Interface down or detached

01:09:08: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from LOADING to FULL,
Loading Done

01:09:08: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/2 from LOADING to FULL,
Loading Done

?Bad filename
%Error parsing filename (Bad file number)
R1#show ip route ospf
     10.0.0.0/8 is variably subnetted, 6 subnets, 2 masks
O       10.0.23.0 [110/2] via 10.0.12.2, 00:06:10, GigabitEthernet0/1
                  [110/2] via 10.0.13.2, 00:06:10, GigabitEthernet0/2
O       10.0.24.0 [110/2] via 10.0.12.2, 00:06:10, GigabitEthernet0/1
O    192.168.30.0 [110/2] via 10.0.13.2, 00:28:57, GigabitEthernet0/2
O    192.168.40.0 [110/3] via 10.0.12.2, 00:06:00, GigabitEthernet0/1

R1#
```

Figure 25: *R1 console: OSPF routes (metric 2 to the server LAN, 3 to the branch LAN), then the Gi0/2 shutdown and restore used for the failover test*

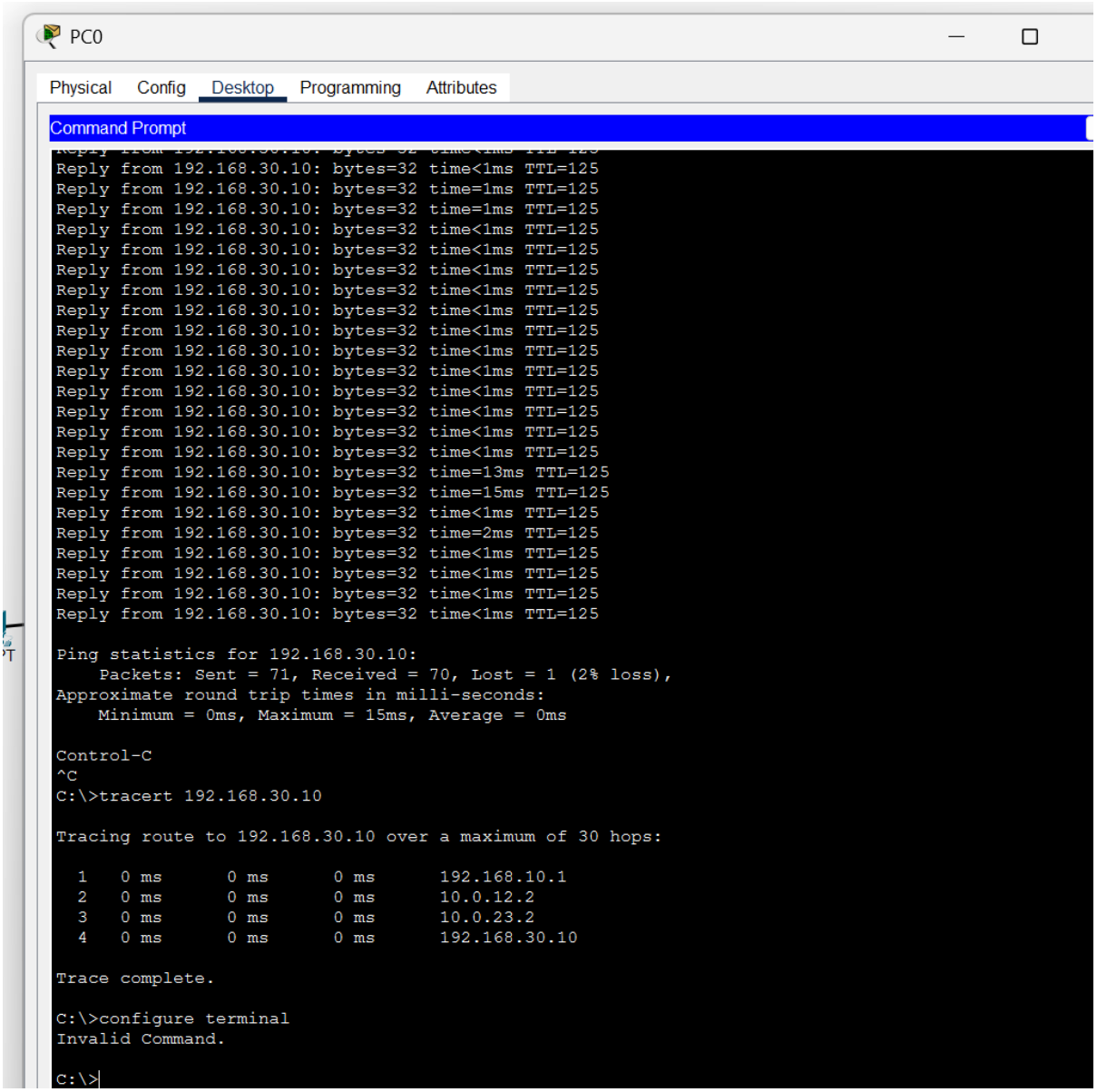


Figure 26: *PC0 during the failover test: 70 of 71 replies, then tracert showing the longer path through R2 and R3*

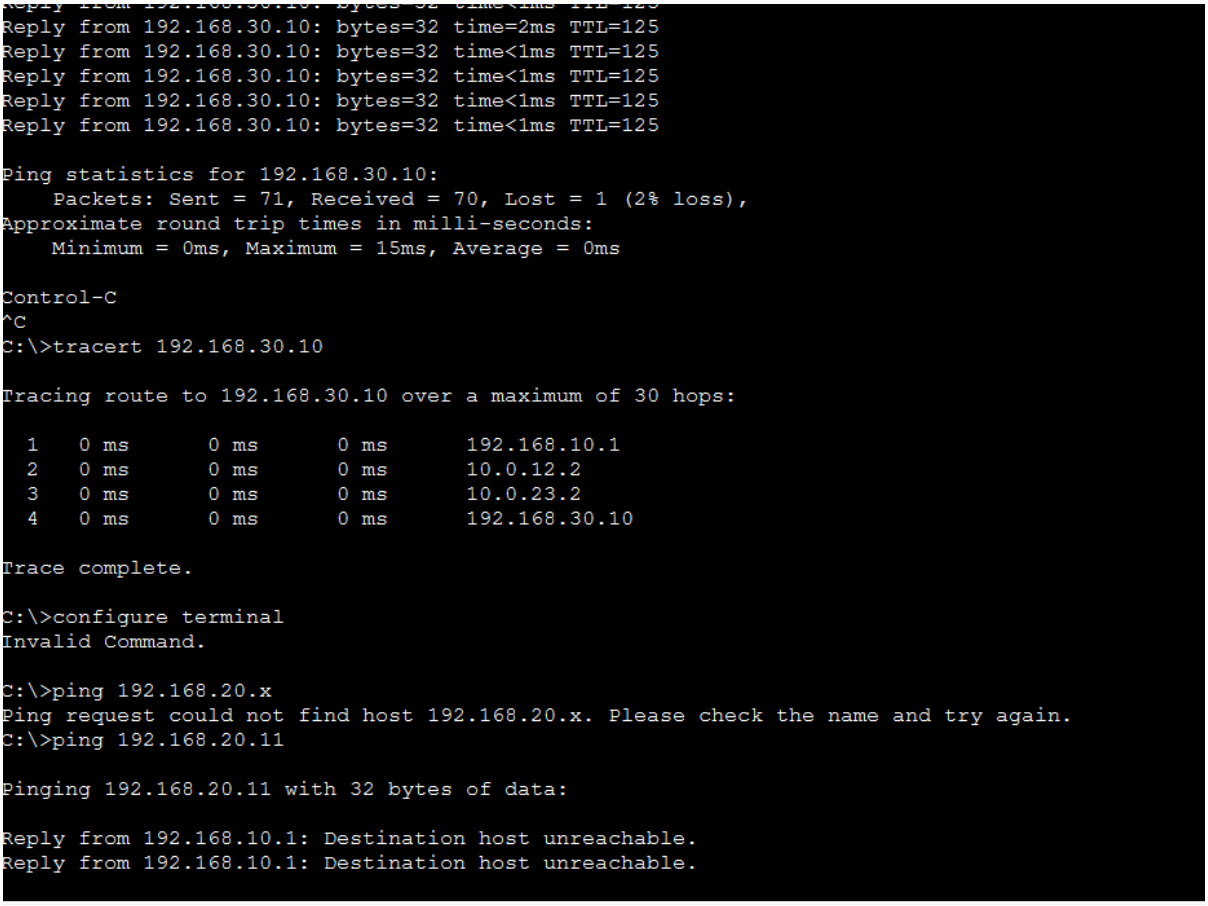


Figure 27: *PC0 pinging the staff PC (192.168.20.11): blocked by the access list on R1*

Test 6 is the convergence measurement. When an interface is shut down, the router detects the loss at once and OSPF recomputes its routes within seconds; a failure that the interface does not detect would instead wait for the 40-second OSPF dead interval. Here one echo reply was lost out of 71. The relevance to the transport layer is that the failover path is one hop longer, so the round-trip time of a running RUDP flow increases and the flow must re-learn its window. Smart-RUDP observes exactly this signal through its RTT-ratio state.

Packet Tracer is a simulator: its devices cannot be captured by Wireshark, and the transport tests of Sections 5 to 8 do not run inside it. The Packet Tracer part of the project therefore validates the network design, and the Wireshark part (Section 10) validates the protocol traffic.

10. WIRESHARK PACKET ANALYSIS

Two kinds of capture are used. The first is a real capture of the loopback interface with tshark inside WSL (Section 6.3). The second is a trace exported by Smart-RUDP itself (the --pcap option), which records every data segment and acknowledgment that reaches its destination across the emulated path, as seen at the receiver, in the standard pcap format. The exported file uses the topology addresses (192.168.10.10 to 192.168.30.10, UDP port 8888) so that it reads as a capture on the server LAN of Figure 23. The headers in this file are written by the program

and not by a network interface, and the connection handshake bypasses the emulator and is not included.

A Wireshark dissector written in Lua (rudp.lua) decodes the 13-byte RUDP header into sequence number, acknowledgment number, flags (SYN, ACK, FIN, DATA), checksum and payload length. Useful display filters are smartrudp.flags.data == 1 for data, smartrudp.flags.ack == 1 for acknowledgments, and smartrudp.seq == N, which lists every copy of one packet and so exposes retransmissions.

Figure 28 is a real capture of the client and server on the loopback interface (rudp_capture_loss.pcapng, 164 packets) taken with tshark while packet loss was applied, and opened in Wireshark with the dissector. The first three packets are the three-way handshake (SYN, SYN-ACK, ACK), and the expanded header of the SYN shows the sequence number, acknowledgment number, flags (0x01), checksum and payload length decoded field by field. In Figure 29, the filter smartrudp.seq == 1327198699 shows the same data segment twice: once at 0.0004 s and again at 0.501 s. The segment before it was lost, so the receiver kept acknowledging the same number, the sender timed out after 500 ms and resent the whole window (Go-Back-N), which is the retransmission behaviour described in Section 3.4.

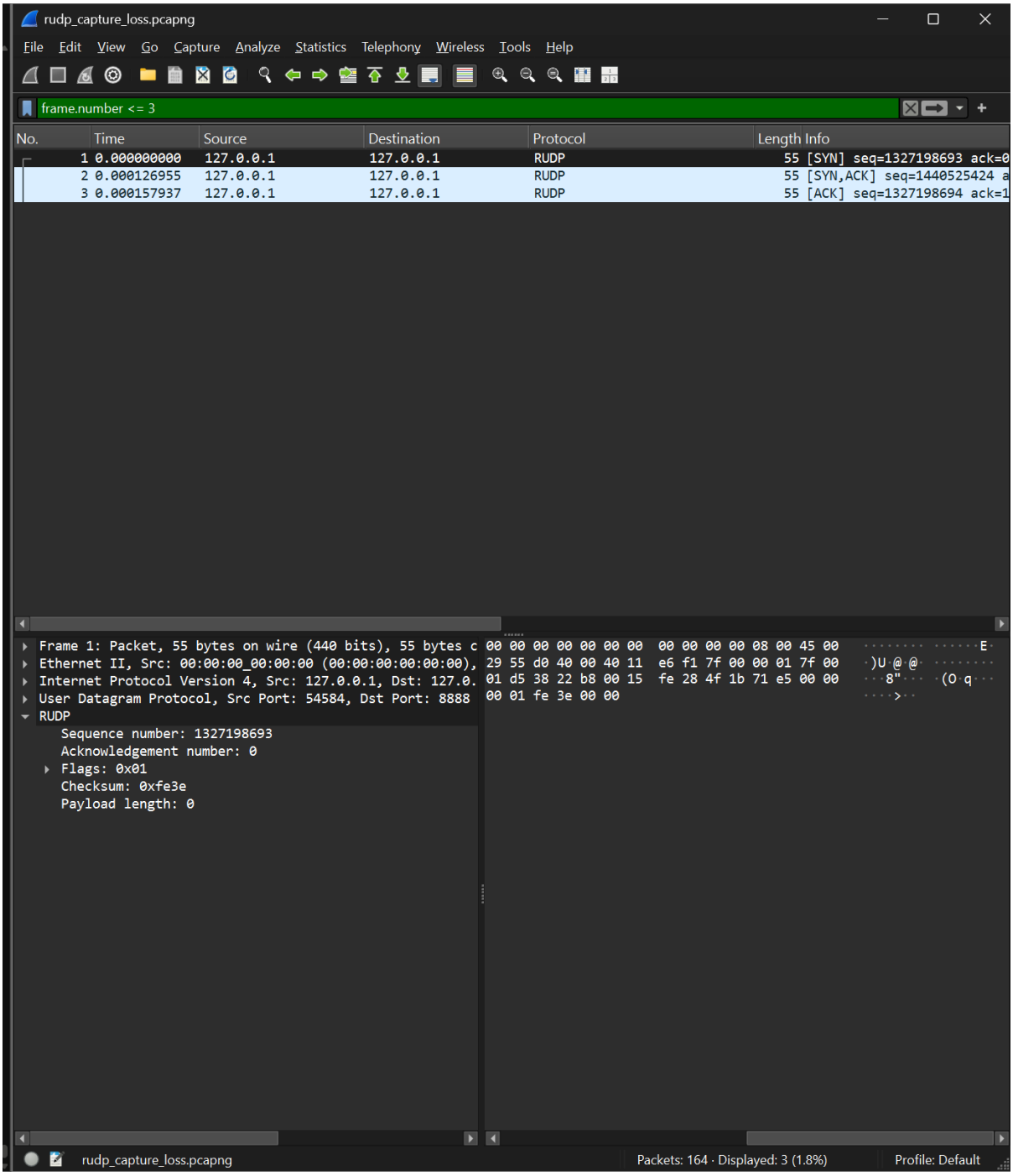


Figure 28: *Real loopback capture opened with the RUDP dissector: SYN, SYN-ACK and ACK, with the header of the SYN decoded*

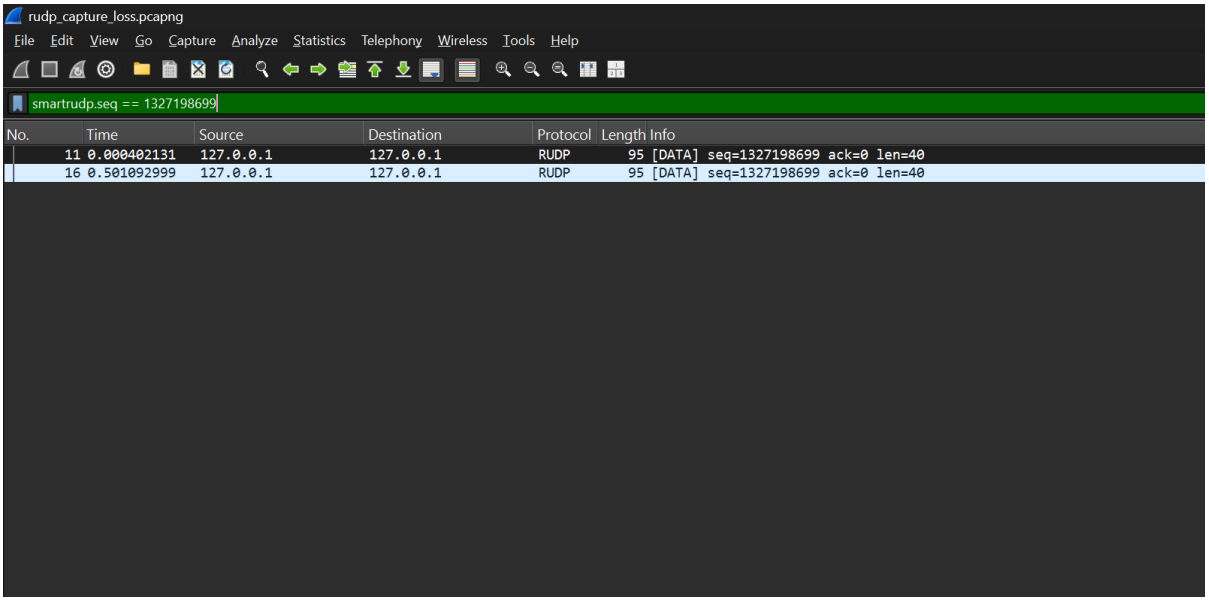


Figure 29: *Display filter smartrudp.seq == 1327198699: the same data segment sent at 0.0004 s and resent at 0.501 s after a timeout*

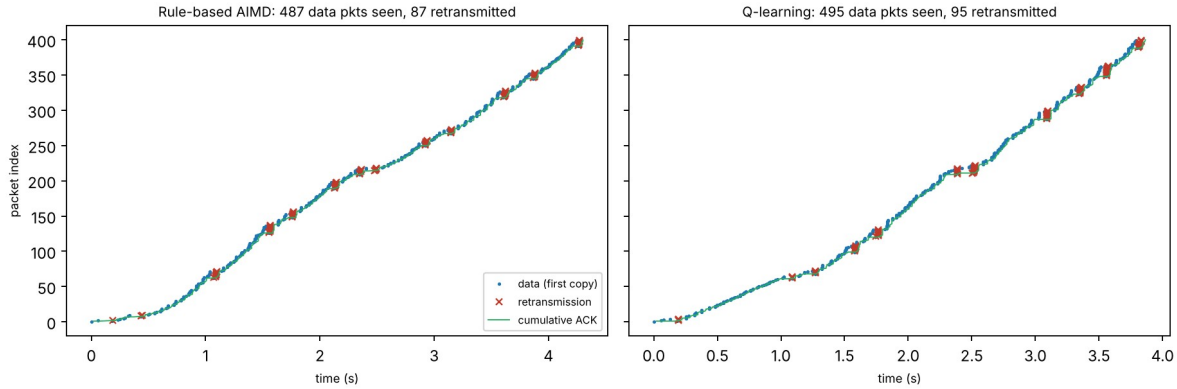


Figure 30: *Sequence-against-time view built from the exported traces of a 400-packet transfer over the lossy-wireless path: first copies of data, retransmissions and the cumulative ACK for AIMD (left) and Smart-RUDP (right)*

Figure 30 plots the two traces in the style of a Wireshark time-sequence graph. In this run the receiver saw 487 data segments and 487 acknowledgments for the AIMD flow, of which 87 data segments carried a sequence number already seen, and 495 data segments and 495 acknowledgments for the Smart-RUDP flow, of which 95 were repeats. The staircase of repeated points shows Go-Back-N resending a window after each loss. The two flows delivered the same 400 packets; the counts are from a single run, are counted at the receiver (segments lost on the path are not visible), and are not a statistical comparison.

11. APPLICATION DEMONSTRATION: RUDP CHAT

To show what the protocol does for a user, a messenger was built on the same packet format in three forms. The terminal program (src/chat.c) is a symmetric peer: each message is one data segment with a sequence number, the receiver delivers messages strictly in order and answers

with cumulative ACKs, and the sender keeps a Go-Back-N window of eight messages with a 300 ms timeout and fast retransmit on three duplicate ACKs. Packet loss is simulated on the sending side, so a user can watch a lost message being recovered while the conversation still arrives complete and in order.

The program was then extended to run between two machines. The peer is given as ip:port, the socket binds to all interfaces, and each side sends small acknowledgment packets to the other until the first packet arrives, which opens the return path through home routers (NAT); the peer address is then learned from the first valid packet. The two ends were connected over a Tailscale private network, one on the laptop and one on an Android phone.

Finally, a browser front end was added (web/rudp_web.py and web/index.html). A browser cannot send raw UDP, so a small gateway written in Python with only the standard library implements the same RUDP rules and serves the chat page over local HTTP. It interoperates with the C program: in a test with 30% simulated loss in both directions, messages from the browser and from the terminal peer all arrived once and in order. The page, shown in Figure 31, marks each sent message as delivered once it is acknowledged and shows a live loss slider, counts of packets sent, lost and resent, the number of messages in flight, and a log of timeouts and fast retransmits.

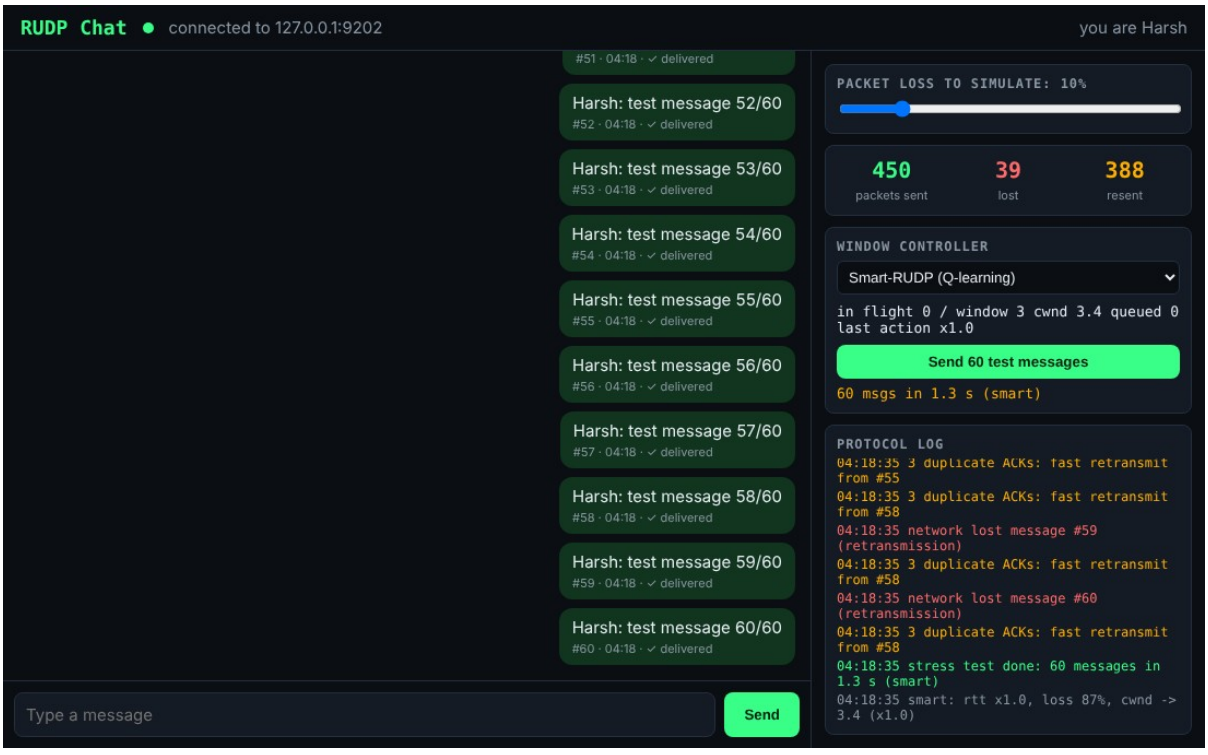


Figure 31: *RUDP Web Chat with the Smart-RUDP controller selected, after a 60-message test at 10% simulated loss and 40 ms emulated delay: delivered ticks, packet counters, window controller and protocol log.*

The web page also lets the user choose the window controller: a fixed window of eight, the AIMD rule, or the Smart-RUDP Q-learning agent. A button sends 60 test messages and reports

how long delivery took. The chat is a different setting from the experiments of Section 8: traffic comes in short bursts and the loss on the path is random rather than caused by a full queue. The Smart-RUDP agent for the chat was therefore retrained (web/train.py) on a discrete-event simulator of the chat transport (web/sim.py), over 6,000 randomly drawn paths with 0 to 15% loss, 20 to 150 ms one-way delay and an optional bottleneck link. Two choices are worth stating. First, AIMD and Smart-RUDP start from the same initial window of 10 (RFC 6928), so neither gets a head start. Second, the agent may not shrink the window while there is neither loss nor delay growth; with this rule removed the trained agent was slower than AIMD on every path, so it is part of the design and not a detail. The agents were evaluated frozen and greedy on six held-out paths and 60 random seeds each; Table 11 gives the mean time to deliver 60 messages.

Table 11: Mean time to deliver 60 chat messages on simulated held-out paths (60 seeds each; lower is better)

Path (one-way delay)	Fixed 8	AIMD	Smart-RUDP	Smart vs AIMD
10% loss, 40 ms	1.09 s	2.17 s	0.90 s	faster
2% loss, 100 ms	1.77 s	1.66 s	1.43 s	faster
6% loss, 60 ms	1.34 s	1.81 s	1.08 s	faster
4% loss, 50 ms, bottleneck	1.30 s	1.55 s	1.37 s	faster (marginal)
No loss, 30 ms, bottleneck	0.61 s	0.56 s	0.56 s	equal
14% loss, 90 ms	2.21 s	4.89 s	1.85 s	faster

Stress test. To check that Table 11 was not a lucky choice of paths, web/stress.py draws 48 random paths the agent never trained on (loss 0 to 30%, delay 10 to 300 ms, jitter up to 80 ms, optional bottleneck of 60 to 400 packets per second with a queue of 5 to 40, 10 to 120 messages) and runs each with 3 seeds. Mean delivery time was 3.73 s for Smart-RUDP, 3.88 s for the fixed window and 7.16 s for AIMD. Smart-RUDP was faster than AIMD on 27 paths, slower on 5 and within 5% on 16. Against the fixed window it was faster on 26, slower on 13 and within 5% on 9. Its weak spots are tight bottleneck queues and very high loss, where the fixed window of eight is hard to beat. A second agent trained on a broader range of paths did worse (mean 5.84 s) and was discarded. These results are from simulation, and Section 8.7 shows that this simulator flattered the learned agents at high loss. In a live test, 200 messages sent at 30% loss in both directions arrived complete and in order in 4.3 s, and the gateway kept responding when the peer disappeared.

Deep reinforcement learning. The abstract proposed a deep agent, so a neural version was added (web/dqn.py, web/train_deep.py). It is a Deep Q-Network written in numpy: a network

with two hidden layers of 32 units reads five continuous inputs (RTT growth, loss fraction, window size, throughput share and the last action) and outputs a value for each of the same five window actions. It learns from a replay buffer with a separate target network, using the same reward, the same no-shrink rule and the same 6,000 simulated training paths as the table agent. Training is not stable: of four random seeds, three produced a good agent and one (seed 12) was slower than AIMD. The agent used here was therefore chosen by its score on 16 separate validation paths, which are not the held-out paths and not the stress paths. On the six held-out paths of Table 11 it was faster than the table on five and equal on one (40 seeds each). On the 48 unseen stress paths its mean delivery time was 3.20 s, against 3.73 s for the table, 3.88 s for the fixed window and 7.16 s for AIMD; it was faster than the table on 19 paths and slower on 6, and faster than AIMD on 27 and slower on 5. The gain over the table is modest, which fits a small state space, and the deep agent is weak on the same tight bottleneck queues as the table agent. It runs in the Python chat gateway and the simulator, and was later added to the C transport (Section 8.5). The simulated figures in this paragraph come from a simulator that allowed a retransmission storm; Section 8.7 repeats them with the realistic rule, where the agent is about level with AIMD, and tests the agent in C with equal caps, where it still wins.

Baselines: plain UDP and TCP. The abstract also proposed TCP and plain-UDP baselines. Plain UDP and a model of TCP were run on the same 48 stress paths (web/baselines.py). The TCP model has slow start, congestion avoidance, fast recovery with selective acknowledgments and a retransmission timer after RFC 6298, with the same initial window of 10 and cap of 32 as the chat. It is a model of TCP, not the kernel stack. Plain UDP delivered on average 68% of the messages and delivered every message on only 4 of the 48 paths, which is the reason reliability is needed. Table 12 gives the times.

Table 12: 48 unseen random paths, 3 seeds each, simulated, 60 messages unless the path says otherwise

Transport / controller	Mean time (s)	Median time (s)	Note
Plain UDP	n/a	n/a	68% delivered
TCP model (selective ACK)	4.56	1.48	Go-Back-N not used
RUDP, fixed window 8	3.88	2.00	
RUDP, AIMD	7.16	2.72	
RUDP, Smart-RUDP table	3.73	1.99	
RUDP, Smart-RUDP deep	3.20	1.79	

The TCP model was faster than RUDP with AIMD on 33 of the 48 paths, and its median is the lowest of all. Its mean is higher than the learned controllers because its timer backs off at very high loss, where RUDP's fixed 300 ms timer does not. The deep agent was faster than the TCP

model on 22 paths and slower on 23. The main reason for the gap to AIMD is Go-Back-N, which resends the whole window after a loss. A basic selective-acknowledgment variant of the same simulated transport (web/sack_test.py) cut the mean time with the fixed window from 3.88 s to 2.53 s and with AIMD from 7.16 s to 4.90 s (median 2.72 s to 1.44 s), and was faster on 39 and 34 of the 48 paths respectively. Selective acknowledgment is therefore the most valuable next change to the protocol. One regime goes the other way: at 10 to 20% random loss the whole-window resend of Go-Back-N acts as redundancy and the fixed window was slightly faster without SACK.

Reordering. The C emulator keeps packets in order, so reordering was tested in the simulator by adding jitter larger than the packet spacing (2% loss, 50 ms delay, 60 messages, 40 seeds; web/reorder_test.py). A Go-Back-N receiver drops out-of-order packets and sends duplicate ACKs, so reordering looks like loss. AIMD with Go-Back-N went from 0.82 s with no jitter to 6.79 s at 20 ms jitter. The learned agents were more tolerant, and selective acknowledgment removed most of the damage (Table 13). The learned agents were trained only on in-order paths with Go-Back-N.

Table 13: Mean time to deliver 60 messages (s) under reordering; GBN is Go-Back-N, SACK is selective acknowledgment; simulated

Jitter	AIMD, GBN	AIMD, SACK	Table, GBN	Deep, GBN	Deep, SACK
0 ms	0.82	0.77	0.77	0.49	0.48
20 ms	6.79	1.61	1.25	0.93	0.64
50 ms	7.06	1.82	1.67	1.13	0.80
100 ms	8.16	2.61	2.77	1.63	1.11
200 ms	10.06	6.34	4.23	2.54	1.55

Smart-RUDP was faster than AIMD on five of the six paths and equal on the sixth, and faster than the fixed window on five paths. The reason is specific: AIMD resets its window to one packet after every timeout, which is expensive when loss is random and not caused by congestion, while the agent learned to hold its window in that case. The advantage is therefore largest on high-loss paths and absent on a clean bottleneck, where there is nothing to learn. These are simulated paths. A real-time check of the gateway with two copies on one machine (four runs per controller) gave a mean of 1.2 s for Smart-RUDP against 2.6 s for AIMD at 10% loss and 40 ms, and 2.3 s against 3.0 s at 2% loss and 100 ms, with the fixed window at 0.8 s and 1.8 s; the run-to-run spread there is large, so those numbers support the direction and not the size of the effect. Section 8 stands as reported: on the 5G-like, lossy and satellite-like paths of that experiment the learned controller did not beat AIMD.

This demonstration is an application of the protocol, not a new measurement. Over the internet the result also depends on the network between the peers: direct connections between two home networks usually work, but shared or carrier-grade NAT, as on many hostel networks, can block them, which is why the private-network route was used.

12. LIMITATIONS

- **Original client (Sections 3 to 6).** The benchmarked 62-packet client uses Go-Back-N with a fixed 500 ms timeout, no adaptive timer and simplified fast recovery (the window is halved on three duplicate ACKs, without the window inflation of RFC 5681). The default transport used from Section 8.9 on is Selective Repeat with an adaptive timer.
- **No kernel network.** The path is emulated in the client process, in a user-space proxy on the loopback interface, or in virtual time. These agree with each other. The comparison was repeated once on kernel tc/netem over loopback on one laptop (Section 8.17, five runs per cell), where kernel TCP is faster than the neural agent on every path; no physical link or multi-machine path was tested.
- **Not a polite neighbour by default.** On a shared bottleneck the neural agent takes 68% to 70% of the link against tuned AIMD and 76% against the original AIMD (Sections 8.13 and 8.15); against a kernel TCP flow it takes less than half (24% to 34%, Section 8.22). The optional polite mode brings this to about half, at a cost in speed and with one path 2.95 times slower; it is a rule added by hand (training the agent for fairness against a simulated competitor did not work, Section 8.15), and coexistence with kernel TCP was measured only for three runs per path on one laptop (Section 8.22).
- **No gain on very lossy paths.** On the path with 10% random loss the agent is not significantly different from AIMD, and the AIMD hand-over exists because the plain agent collapsed there under Go-Back-N (Section 8.8). The safety net is a heuristic, not a guarantee.
- **Training data.** The agent was trained in a Python simulator for windows up to 32; it generalises to a cap of 128 and to the C sender (Section 8.12), and an evolution-strategy fine-tune in the C sender gave no robust gain, but it has never seen a real network trace. With bursty cross traffic (Section 8.15) its advantage over tuned AIMD shrinks to about 15% and it is level with CUBIC.
- **Scale and features.** Payloads are at most 512 bytes and file mode is limited to 32 MB; the server handles up to 16 clients at once but each only one connection; flow control is only the 256-packet receive window; there is no encryption, multiplexing or connection migration (QUIC provides these).
- **Statistics.** Real-clock comparisons use 10 runs per cell (5 on the hostile path) and the proxy comparison 8; the 100-seed and 100-path results are in virtual time. The intervals

on the hostile path are wide and the conclusion there is only that no difference was detected.

13. CONCLUSION AND FUTURE ENHANCEMENTS

RUDP shows that TCP's core guarantees, ordered and complete delivery over a lossy datagram service together with a self-limiting sending rate, can be built and verified from first principles. The final system delivered all 62 packets of a 2,442-byte message at every simulated loss level from 0% to 40%. The congestion window responded as AIMD predicts, shrinking from 18.71 to 2.00 packets as loss rose, and the 500 ms retransmission interval was visible both in the cwnd graph and in the packet capture.

Smart-RUDP then tested whether a learned window policy could improve on the rule. A tabular Q-learning agent matched AIMD on the lossy path and was slower on the 5G-like and satellite-like paths; the honest finding is that this simple agent does not beat a well-tuned rule. A small neural agent did. In the final configuration (Selective Repeat, window cap 128, slow start to 64 packets, pacing, a hand-over to AIMD at 10% loss, and a retransmission-timer floor that every policy uses) it needs 15% to 53% less time than a tuned AIMD baseline on four ordinary paths, measured with a real clock, on real sockets through an impairment proxy, and in a virtual-time C sender that reproduces the real runs; on 100 random held-out paths it needs 0.53 times the time and is never more than 20% slower. It beats tuned CUBIC on the lossy and satellite-like paths and equals it elsewhere, it ties AIMD on a 10% loss path, and it takes more of a shared link than AIMD does. Several results went against the first impression and are reported as such: the apparent advantage was inflated by a timer defect that hurt the baseline more, the plain agent failed under heavy loss with Go-Back-N, a timeout cut and a retrained agent were dropped, and an evolution-strategy fine-tune gave no robust gain. The network design adds a four-router OSPF network with VLANs, DHCP, an access list and a tested link failover in Cisco Packet Tracer, and the Wireshark work adds a dissector and traces that make the protocol behaviour visible packet by packet.

The project also taught that reliable transport is largely timers and edge cases, that congestion control is a feedback loop in which the sender listens to the network, that independent evidence such as a packet capture matters, and that debugging the environment is as important as debugging the code.

Future enhancements are:

- Repeating the kernel comparison on a real multi-machine path, with more runs per cell, and closing the gap to kernel TCP (a larger initial window and cheaper per-packet handling are the first candidates).
- Making the agent a fairer neighbour by learning rather than by rule: add a competing flow to the training environment and a fairness term to the objective (the polite mode of

Section 8.15 is the hand-made version), then repeat the coexistence test with more runs and with other kernel congestion controls.

- Training on recorded network traces and with cross traffic, and widening the state beyond five inputs.
- A learned policy that decides when to hand over, instead of the 10% loss rule of Section 8.8.
- Comparing against the real Linux BBR (with ProbeRTT and, for BBRv3, its loss response) instead of the simplified policy of Section 8.19, and adding an explicit-congestion-notification signal as an extra input.
- Training with a fairness objective and with formal certificates in mind (Section 8.21 shows that the current network can be certified only at tiny input perturbations).
- Encryption, multiplexed streams and connection migration, which would turn the transport into a QUIC-like library, and packaging it with a stable API.

REFERENCES

[1] V. Jacobson and M. J. Karels, "Congestion Avoidance and Control," in Proc. ACM SIGCOMM '88, 1988, pp. 314-329.

[2] D.-M. Chiu and R. Jain, "Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks," Computer Networks and ISDN Systems, vol. 17, no. 1, pp. 1-14, 1989.

[3] M. Allman, V. Paxson and E. Blanton, "TCP Congestion Control," RFC 5681, IETF, Sept. 2009.

[4] S. Ha, I. Rhee and L. Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant," ACM SIGOPS Operating Systems Review, vol. 42, no. 5, pp. 64-74, July 2008.

[5] A. Langley et al., "The QUIC Transport Protocol: Design and Internet-Scale Deployment," in Proc. ACM SIGCOMM '17, Los Angeles, CA, Aug. 2017.

[6] J. Iyengar and M. Thomson, Eds., "QUIC: A UDP-Based Multiplexed and Secure Transport," RFC 9000, IETF, May 2021.

[7] S. Hemminger, "Network Emulation with NetEm," Open Source Development Lab, Apr. 2005, as cited in the tc-netem manual page.

[8] J. Postel, "User Datagram Protocol," RFC 768, IETF, Aug. 1980.

[9] K. S. Midhula and P. Arun Raj Kumar, "An Adaptive Congestion Control Protocol for Wireless Networks Using Deep Reinforcement Learning," IEEE Trans. Network and Service Management, vol. 21, no. 2, pp. 2027-2043, 2024.

[10] Y. Cao, J. Nie, Y. Fan, X. Shao and G. Lei, "DRLFcc: Deep Reinforcement Learning-empowered Congestion Control Mechanism for TCP Fast Recovery in High Loss Wireless Networks," in Proc. IEEE GLOBECOM 2023, pp. 4345-4350, 2023.

[11] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3-4, pp. 279-292, 1992.

[12] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey and M. Schapira, "PCC: Re-architecting Congestion Control for Consistent High Performance," in Proc. USENIX NSDI 2015, pp. 395-408.

[13] J. Moy, "OSPF Version 2," RFC 2328, IETF, Apr. 1998.

[14] A. Mazilu, B. Giacomoni and I. Parisis, "Learning-Based vs Human-Derived Congestion Control: An In-Depth Experimental Study," arXiv:2510.25105, 2025.

[15] Tian et al., "Fairness generalizability of learned congestion control" (Jury), in Proc. ACM EuroSys 2025. [Details to be checked by the authors against the published paper.]

[16] Yang et al., "C3: Learning Congestion Controllers with Formal Certificates," arXiv:2412.10915, 2024.

[17] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh and V. Jacobson, "BBR: Congestion-Based Congestion Control," ACM Queue, vol. 14, no. 5, 2016.